Computer Physics Communications 273 (2022) 108266

Contents lists available at ScienceDirect

Computer Physics Communications

www.elsevier.com/locate/cpc

QTRAJ 1.0: A Lindblad equation solver for heavy-quarkonium dynamics ☆,☆☆

Hisham Ba Omar [a], Miguel Ángel Escobedo [b], Ajaharul Islam [a], Michael Strickland [a,*], Sabin Thapa [a], Peter Vander Griend [c], Johannes Heinrich Weber [d]

[a] Department of Physics, Kent State University, Kent, OH 44242, United States
[b] Instituto Galego de Física de Altas Enerxías (IGFAE), Universidade de Santiago de Compostela, E-15782, Galicia, Spain
[c] Physik-Department, Technische Universität München, James-Franck-Str. 1, 85748 Garching, Germany
[d] Institut für Physik, Humboldt-Universität zu Berlin & IRIS Adlershof, D-12489 Berlin, Germany

ARTICLE INFO

Article history:
Received 23 July 2021
Received in revised form 10 August 2021
Accepted 14 December 2021
Available online 24 December 2021

Keywords:
Heavy quarkonium
Open quantum systems
Lindblad equation
Quantum trajectories

ABSTRACT

We introduce an open-source package called `QTraj` that solves the Lindblad equation for heavy-quarkonium dynamics using the quantum trajectories algorithm. The package allows users to simulate the suppression of heavy-quarkonium states using externally-supplied input from 3+1D hydrodynamics simulations. The code uses a split-step pseudo-spectral method for updating the wave-function between jumps, which is implemented using the open-source multi-threaded FFTW3 package. This allows one to have manifestly unitary evolution when using real-valued potentials. In this paper, we provide detailed documentation of `QTraj` 1.0, installation instructions, and present various tests and benchmarks of the code.

Program summary
Program Title: QTraj 1.0
CPC Library link to program files: https://doi.org/10.17632/34hbsb6pzy.1
Developer's repository link:: https://bitbucket.org/kentphysics/qtraj-fftw
Licensing provisions:: GPLv3
Programming language:: C
Nature of problem:: Solution of the Lindblad equation for evolution of the heavy-quarkonium reduced density matrix in a quark-gluon plasma.
Solution method:: We use the quantum trajectories algorithm to solve the Lindblad equation.
© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Quantum Chromodynamics (QCD) is the quantum field theory that describes the strong nuclear force. The degrees of freedom contained in the QCD Lagrangian are quarks and gluons, which are collectively called partons. At low temperatures ($T \lesssim 2 \times 10^{12}$ K), partons are confined inside hadrons such as protons and neutrons, and, as a result, they are never observed in isolation. At temperatures higher than this threshold or, alternatively, at sufficiently high baryon density, QCD predicts a phase transition to a deconfined quark-gluon plasma (QGP) [1–3]. The QGP can be studied on Earth by colliding ultrarelativistic heavy ions, such as lead or gold, with ongoing experiments being performed at both the Relativistic Heavy Ion Collider at Brookhaven National Laboratory and the Large Hadron Collider at the European Organization for Nuclear Research (CERN) [4,5].

In practice, it is very challenging to extract dynamical information about the QGP from experimental data since this new state of matter exists for only a very short time after the collision, $t_{\rm QGP} \lesssim 15$ fm/c. It was theorized in [6,7] that heavy quarkonium, a colorless bound state consisting of a heavy quark and a heavy anti-quark, melts inside of the QGP and, therefore, quarkonium suppression in heavy-ion collisions could be used as an important signature of the formation of a QGP. The theoretical description of in-medium quarkonium dynamics has changed significantly since these early studies. In the beginning, it was thought that the main phenomenon responsible for quarkonium suppression was color

☆ The review of this paper was arranged by Prof. Z. Was.
☆☆ This paper and its associated computer program are available via the Computer Physics Communications homepage on ScienceDirect (http://www.sciencedirect.com/science/journal/00104655).
* Corresponding author.
E-mail address: mstrick6@kent.edu (M. Strickland).

https://doi.org/10.1016/j.cpc.2021.108266
0010-4655/© 2021 Elsevier B.V. All rights reserved.

screening. Later, however, other phenomena were identified, which are equally or more important [8–12]. In particular, one finds that collisions between heavy-quark bound states and particles in the medium introduce dispersion, which results in an imaginary contribution to the heavy-quark potential [8,13–15]. In fact, in the small coupling limit, collisions dissociate quarkonium bound states already at small distances where screening is not yet relevant. In addition to collisional dissociation, one must also consider recombination [16,17], a process in which a heavy quark and anti-quark, which could be initially uncorrelated, form a new bound state inside the medium or during late-time hadronic freeze-out. Statistical recombination is potentially very important for charm quarks, as opposed to bottom quarks, since charm quarks are more copiously produced in heavy-ion collisions [18]; however, local quantum regeneration could also play a role.

As a result, we need a theoretical description that can self-consistently include all three of these mechanisms. In recent years, there have been many advances in the application of the open quantum system formalism to heavy quarkonium dynamics [19–33]. In this approach, information about the quantum state of quarkonium is encoded in a reduced density matrix and the equation that describes the evolution of the reduced density matrix is called a master equation. In this paper, we focus on the master equations developed in [24,25]. Therein, it was found that, if the temperature T is much larger than the binding energy E, the master equation is Markovian. It is a well-known result in the field of open quantum systems that all Markovian evolution equations that conserve the properties of the density matrix (trace-preserving, etc.) are Lindblad equations [34,35]. In [24,25], the Lindblad equation was solved numerically in order to obtain phenomenological results. This was done in the following way:

- Expanding the density matrix in spherical harmonics and truncating at a given maximum angular momentum. In this way, the dimensionality of the problem is reduced from three to one dimensions.
- Discretizing the one dimensional space on a finite lattice.
- Using the *mesolve* function of the publicly available package QuTiP 2 [36,37].

However, as the size of the angular momentum basis and one-dimensional lattice used to solve the Lindblad equation become large, a huge amount of computational resources are required. For example, a naive counting indicates that doubling the size of the lattice increases the computational cost by a factor of at least four. This is a well-known problem faced when solving the Lindblad equation, and there is a large literature on stochastic methods that can be used to reduce the computational cost [38–43]. One of the more popular approaches is the Monte-Carlo wave-function or Quantum Trajectories method, originally developed for applications in quantum optics [38]. In [44], this method was applied to quarkonium phenomenology and resulted in a substantial reduction in the computational cost. In addition to the reduction in computational cost, another advantage of the quantum trajectories algorithm is that this method does not require truncation in angular momentum. As we will discuss, the most computationally costly operation that appears in the application of the quantum trajectories algorithm to the study of quarkonium suppression is the real-time solution of the Schrödinger equation with a non-Hermitian Hamiltonian. Luckily, there is existing literature concerning optimization of this aspect of the calculation which can be directly applied to this problem [45–47].

In [44], in order to maximize code efficiency, the computations were performed using graphical processing units (GPUs). In this paper, we introduce a CPU-based code in order to allow the code to be run more easily by practicing scientists. Associated with this paper, the CPU-based code is being made publicly available so that it can be used by others to study in-medium quarkonium suppression and, with some adjustments, similar problems. As detailed below, we achieve high efficiency by combining the quantum trajectories algorithm with a split-step pseudo-spectral method to compute the evolution of the wave-function subject to a non-Hermitian Hamiltonian. We also introduce several optimizations of the code which make the CPU code comparable in speed to the GPU code. We denote the resulting code `QTraj`.

The paper is organized as follows. In Section 2, we describe how to download, compile, and run `QTraj`. In Section 3, we detail the runtime parameters of the code. In Section 4, we provide a brief theoretical background and describe the algorithm as applied to the Lindblad equation studied in [44]. Key parts of the code are discussed in Section 5. In Section 6, we provide instructions for how to analyze `QTraj` output. In Section 7, we present tests of the code and perform scaling analyses to determine the dependence on the parameters such as lattice size, etc. In Section 8, we benchmark the code and compare to matrix-based methods for solving the Lindblad equation. Finally, in Section 9, we present our conclusions. In the appendices we collect a list of required packages, details concerning the input/output file formats, and the potentials which are implemented.

2. Download and compilation of the code

2.1. How to download the code

To download the most up-to-date version of `QTraj`, the user can either pull from the public GIT repository

```
$ git clone https://bitbucket.org/michael_strickland/qtraj-fftw.git
```

or download tagged versions using the URL contained in [48].

2.2. Structure of the `QTraj` package

After download, the `QTraj` package main folder has the following top-level directory structure

```
qtraj-fftw
├── build
├── docs
├── input
├── license
├── mathematica
├── scripts
└── test
```

2.3. Compilation of `QTraj` and unit tests

Once the requisite packages listed in Appendix A have been installed, `QTraj` can be compiled by simply typing

```
$ make
```

from the main directory. The user can compile with all cores on their system by adding '`-j`' or specify a given number of cores by adding '`-j #`'. Upon successful compilation, an executable called `qtraj` will be generated in the main folder. The Makefile will automatically detect if the user has MKL installed. If MKL is installed, the MKL version of the internal eigensolver will be used, otherwise, the Armadillo version will be used. To reset the build,

the user can execute `make clean` from the main directory. Note that it is possible to compile the code without an internal eigensolver by executing `make mode=lightweight` from the main directory. In this case, runtime numerical determination of the eigenstates is not possible; however, the user can read previously computed eigenstates from disk (see Sec. 3.1 for more details).

After compiling the main executable, the user can run unit tests on the code by executing

```
$ make tests
```

from the main directory. Note that this functionality requires the GoogleTest framework to be installed. See Appendix A for further information.

To run `QTraj` using the parameters from `input/params.txt`, the user executes

```
$ ./qtraj
```

in the directory where the code was compiled. All output from the code will appear either in a directory called `output` or `output-<random-seed>`, depending on the output parameter settings.

3. Code parameters

All runtime parameters for the code can be set by editing the file `input/params.txt`. We note that `QTraj 1.0` does not implement consistency checks on the user supplied values of the runtime parameters. Parameter values set from the file can be overridden by appending a dash followed by the name of the parameter and the value to use, e.g.

```
$ ./qtraj -nTrajectories 16
```

sets the value of `nTrajectories` to 16, overriding the value indicated in the file `input/params.txt`. In Table 1, we provide documentation for each runtime parameter.[1]

3.1. Using the built-in eigensolvers

`QTraj` includes the ability to determine the vacuum eigenstates numerically based on the potential set using the `potential` parameter. For this purpose, it makes use of either the Intel MKL library [49] or the Armadillo library [50] (see Appendix A for details). To use this feature, the user must set `initType = 100`. Upon code execution, `QTraj` first determines the eigenstates and outputs the masses of the 1S, 2S, 1P, 3S, 2P, and 1D states.[2] These eigenstates are saved to a file in the output folder called `basisfunctions.tsv`. Once generated, this file can be used to initialize the eigenstate basis. To do this, the user should copy the `basisfunctions.tsv` file into the `input` folder, adjust the parameter `basisFunctionsFile` to point to this file, and finally change the parameter `initType` to 200. This is useful for running `QTraj` on machines that do not have either the MKL or Armadillo packages installed.

[1] Some of the parameters describing the two KSU potentials are hard-coded; their description can be found in Appendix E.2 and Appendix E.3.

[2] There is an internal variable `nBasis` that can be adjusted to output more basis states. The set of principal and angular momentum quantum numbers of the basis states have to be adjusted by the user before recompiling the code.

3.2. Derivative types

During the H_{eff} evolution of the wave-function, the `QTraj` code can make use of two different types of kinetic energy operators. For the case `derivativeType = 0`, the momentum-space representation of the kinetic energy operator $T_{\text{kin}} = p^2/m$ is used, where m is the mass of the heavy quark; $m/2$ is the reduced mass in the heavy quarkonium system. This is the default setting in `QTraj`. If the user specifies `derivativeType = 1`, the momentum-space representation of the kinetic energy operator is taken to be of the form $T_{\text{discrete}} = 2[1-\cos(p\Delta r)]/(m\Delta r^2)$, which corresponds to using centered differences to compute the second derivative operator. This option is less accurate than using the full kinetic energy operator. It exists to facilitate comparison to codes using second order differences.

4. Theoretical background and description of the algorithm

4.1. Lindblad equation

For a wide range of quantum-mechanical applications, we are faced with a Markovian time evolution of a density matrix ρ of a subsystem interacting with an environment that preserves the trace of the subsystem's density matrix. Such a system can be described by the GKSL or Lindblad equation [34,35] whose general form is given by

$$\frac{d\rho}{dt} = -i[H,\rho] + \sum_n \left(C_n \rho C_n^\dagger - \frac{1}{2}\{C_n^\dagger C_n, \rho\} \right). \tag{1}$$

H is a Hermitian Hamilton operator that describes deterministic and unitary time evolution of the subsystem in the presence of the environment, whereas the collapse operators C_n describe processes involving explicit exchanges of quanta between the subsystem and the environment. On the level of the subsystem, these lead to non-unitary time evolution. Defining for those non-unitary interactions partial and total widths

$$\Gamma_n = C_n^\dagger C_n\,, \quad \Gamma = \sum_n \Gamma_n\,, \tag{2}$$

we can reformulate the Lindblad equation in terms of a non-Hermitian effective Hamiltonian

$$H_{\text{eff}} = H - \frac{i}{2}\Gamma\,, \tag{3}$$

that gives rise to non-unitary, but deterministic, time evolution and transitions between different quantum states that are driven by the term $C_n \rho C_n^\dagger$. Rewriting the Lindblad equation as

$$\frac{d\rho}{dt} = -iH_{\text{eff}}\rho + i\rho H_{\text{eff}}^\dagger + \sum_n C_n \rho C_n^\dagger\,, \tag{4}$$

we are in a position to discretize the time evolution in a form that is in principle amenable to numerical integration,

$$\rho(t+dt) = \rho(t) - iH_{\text{eff}}\rho(t)dt + i\rho(t)H_{\text{eff}}^\dagger dt + \sum_n C_n \rho(t) C_n^\dagger dt\,, \tag{5}$$

where each discrete step of numerical integration has a diagonal and an off-diagonal component. Due to the linearity of Eq. (1), one may have any pure state as initial condition and average the late time results over different appropriately weighted initial states.

Table 1
Table of `QTraj` runtime parameters. These can be adjusted within the file `input/params.txt` or on the command line (see Sec. 3).

Group	Parameter	Value/Type	Description
Potential parameter	potential	0	Munich potential (Appendix E.1)
		1	Isotropic KSU potential (Appendix E.2)
		2	Anisotropic KSU potential (Appendix E.3)
Jump parameters	doJumps	1	Perform quantum jumps; only for potential=0
		0	Do not perform quantum jumps
	maxJumps	integer ≥ 0	Maximum number of quantum jumps
Physics parameters	m	real ≥ 0	Reduced mass in units of GeV
	alpha	real ≥ 0	Coulomb coupling; $\alpha = C_F \alpha_s$
	kappa	-1	Central fit (Appendix E.1)
		-2	Lower fit (Appendix E.1)
		-3	Upper fit (Appendix E.1)
		real ≥ 0	Coefficient $\hat{\kappa}$ in Munich potential (Appendix E.1)
	gam	real ≤ 0	Coefficient $\hat{\gamma}$ in Munich potential (Appendix E.1)
Temperature parameters	temperatureEvolution	0	Ideal Bjorken evolution; $T(\tau) = T_0(\tau_{\rm med}/\tau)^{1/3}$
		1	Read from temperature evolution file (Appendix C.1)
		2	Read from "trajectory file" (Appendix C.2)
	temperatureFile	string	Specifies temperature file path
	T0	real > 0	Initial temperature in units of GeV (Bjorken)
	Tf	$0 <$ T0 $<$ Tf	Final temperature in units of GeV
	tmed	real ≥ 0	Time to turn on hydro background in units of GeV^{-1}
	t0	real ≥ 0	Time to begin vacuum evolution in units of GeV^{-1}
Initial condition parameters	initType	0	Singlet Coulomb eigenstates
		1	Gaussian delta function
		100	Computed eigenstates (Section 3.1)
		200	Loads eigenstates from file (Section 3.1)
	ProjType	0	Uses coulomb eigenstates
		1	Uses computed eigenstates
		2	Uses disk-based eigenstates (Section 3.1)
	initWidth	real ≥ 0	Initial width for Gaussian IC (initType=1)
	initN	1	Principal quantum number n; only for initType=0
	initL	0	Angular momentum quantum number l
	initC	0	Singlet color configuration
		1	Octet color configuration; only for potential=0
	basisFunctionsFile	string	Specifies basis functions file path (Section 3.1)
Simulation parameters	nTrajectories	integer ≥ 0	Number of quantum trajectories to simulate
	randomseed	0	Uses high-resolution system timer as a seed
		integer > 0	Uses specified number as a seed
		-1	Uses deterministic random numbers (for testing)
	rMax	$0 <$ real ≤ 1	Sets maximum initial random number (Section 4.4)
	maxJumps	integer ≥ 0	Sets maximum number of jumps (Section 4.4)
Grid parameters	num	integer ≥ 2	Number of lattice sites; 2^n for best performance
	L	real > 0	Size of simulation box in units of GeV^{-1}
	dt	real > 0	Time step in units of GeV^{-1}
	maxSteps	integer ≥ 0	Maximum number of time steps
	derivType	0	Full derivative (Section 3.2)
		1	Second-order differences (Section 3.2)
Output parameters	snapFreq	integer ≥ 0	Frequency for the summary output
	snapPts	integer $\leq$ num	For best performance num/snapPts=2^n
	dirnameWithSeed	0	Output directory "output"
		1	Output directory "output-<seed>"
	saveWavefunctions	0	Turns off saving of wavefunctions
		1	Turns on saving of wavefunctions
	outputSummaryFile	0	Turns off output of the summary file summary.tsv
		1	Turns on output of the summary file summary.tsv

4.1.1. First principles description of quarkonium using effective field theory

A heavy quark-antiquark pair coupled to a hot, nuclear medium can be described using the formalism of Open Quantum Systems (OQS) applied to the effective field theory (EFT) potential Non-Relativistic QCD (pNRQCD); we denote this combined approach OQS+pNRQCD. pNRQCD is obtained from QCD by first integrating out the scale of the heavy-quark mass m, leading to Non-Relativistic QCD (NRQCD) [51–53], and then successively performing a multipole expansion in the relative distance r between the heavy quark and antiquark [53,54]. The latter step integrates out the soft scale $1/r \sim mv$ and the resulting EFT describes heavy quark-antiquark pairs in color singlet or octet configurations as well as light quarks and gluons at the ultrasoft scale $\alpha_s/r \sim mv^2$. The Wilson coefficients of pNRQCD are functions of r, and play the role of potentials, although higher order contributions have non-potential form as well. The evolution equation for the heavy quarkonium reduced density matrix can be obtained within pNRQCD and cast into Lindblad form as long as the medium's scales are much smaller than the inverse Bohr radius and much larger than the ground state binding energy [24,25]. For other hierarchies, a Lindblad form cannot be obtained. In anticipation of details that we review later, we mention that the problem has radial symmetry if the medium is isotropic. Hence, the internal quanta that are ex-

changed between the subsystem and the environment relate to the color configuration (singlet or octet) of the heavy quark-antiquark pair and its angular momentum; the deterministic evolution with H_{eff} leaves the color state as well as the angular momentum invariant, while the collapse operators change color and angular momentum in a well defined manner. For a brief overview of the structure, see Section 4.3; for hard-coded numerical values of the parameters, see Appendix E.1.

4.1.2. Phenomenological descriptions of quarkonium

Alternatively, one may use a hard-coded, phenomenologically motivated interaction of the heavy quark-antiquark pair. A KMS potential [7,55,56] with an imaginary part as dictated by the next-to-leading order result in the electric screening regime [8] is provided in its isotropic (see Appendix E.2) or anisotropic (see Appendix E.3) variants. These phenomenological potentials consider only the color singlet and deterministic evolution with H_{eff}; there are no collapse operators implemented in the phenomenological models included in this version. We note that it is straightforward to extend the code to include new phenomenological potentials. In addition, it is possible for users to include collapse terms for phenomenologically motivated potentials.

4.2. The Monte-Carlo wave-function method

Direct evaluation of Eq. (5) is very demanding and necessarily approximate. Besides the obvious need for a sufficiently fine discretization of time, a discretization of the underlying space on a grid with a finite spacing and a finite box size (or decomposition in terms of a finite number of basis functions) is required. If the interaction is isotropic and central, it is sufficient to discretize the radial direction and treat angular momentum as an internal quantum number. Any numerical implementation of $\rho(t)$ necessitates that the Hilbert space be truncated to a finite set of values of each internal quantum number among which the system can fluctuate through successive applications of the collapse operators. In [24,25], Eq. (5) was solved numerically by considering only S- and P-wave states. Because the size of the density matrix grows as N^2, where N is the product of the number of spatial grid points and of the number of internal quantum states, extending the cutoffs quickly makes the numerical integration prohibitively numerically expensive.

As usual, the only known escape from this curse of dimensionality is stochastic integration. In the literature, various stochastic techniques have been developed to tackle this issue, and they go by the name of *master equation unraveling* (see [57] and references therein). One such unraveling, the Quantum State Diffusion [58], has been already applied to the study of quarkonium, see for example the recent papers [59,60].

We use a variant of the Monte-Carlo Wave-Function (MCWF) method [38] that is particularly advantageous for compact in-medium heavy quark-antiquark pairs and makes efficient use of the symmetries of their interactions. In particular, H_{eff} is diagonal in the space of internal quantum numbers. We write the density matrix as a linear combination of projection operators $\rho(t) = \sum_\psi p_\psi(t) P_\psi$, $P_\psi = |\psi\rangle\langle\psi|$, where ψ is a multi-index characterizing the quantum state; in the case of quarkonium, the angular momentum quantum number $l = 0, 1, 2, \ldots$, the internal quantum number (color c, which can be singlet s or octet o which represents the eight equivalent color octet states), and the radial dependence must be specified. Assuming azimuthal symmetry of the isotropic medium and the interaction, we can integrate over the azimuthal angle and omit the magnetic quantum number m. $p_\psi(t) \geq 0$ are the probabilities for finding the system at time t in the state $|\psi(t)\rangle$ and satisfy $\sum_\psi p_\psi(t) = 1$.

As mentioned above, any state of the subsystem at any time t can be understood as a superposition of initial pure states that have been evolved to time t. For this reason, it is sufficient to consider Eq. (5) for the evolution of a pure state $|\psi(t)\rangle$ with density matrix $\rho(t) = |\psi(t)\rangle\langle\psi(t)|$ by one time step dt:

$$
\begin{aligned}
&\rho(t+dt)\\
&= (1 - iH_{\text{eff}}dt)\,|\psi(t)\rangle\langle\psi(t)|\\
&+ i\,|\psi(t)\rangle\langle\psi(t)|\,H_{\text{eff}}^\dagger dt + \sum_n C_n\,|\psi(t)\rangle\langle\psi(t)|\,C_n^\dagger dt\\
&= (1 - \langle\psi(t)|\,\Gamma\,|\psi(t)\rangle\,dt)\\
&\quad\times \frac{(1 - iH_{\text{eff}}dt)\,|\psi(t)\rangle\langle\psi(t)|\,(1 + iH_{\text{eff}}^\dagger dt)}{(1 - \langle\psi(t)|\,\Gamma\,|\psi(t)\rangle\,dt)}\\
&\quad+ \sum_n \langle\psi(t)|\Gamma_n|\psi(t)\rangle dt\,\frac{C_n\,|\psi(t)\rangle\langle\psi(t)|\,C_n^\dagger dt}{\langle\psi(t)|\Gamma_n|\psi(t)\rangle dt} + \mathcal{O}(dt^2)\,. \qquad (6)
\end{aligned}
$$

The expectation value of the total width Γ for the time evolved state $|\psi(t)\rangle$ at time t,

$$\Gamma(t) \equiv \langle\psi(t)|\,\Gamma\,|\psi(t)\rangle \geq 0\,, \qquad (7)$$

can be used to define probabilities for two different modes of evolution in a single time step,[3] either by deterministic, non-unitary evolution, or by a quantum jump. The stochastic pattern of both types of intermittent evolution characterizes each quantum trajectory $|\psi(t)\rangle$.

How many such trajectories $|\psi(t)\rangle$ are required to achieve sufficient statistical coverage of the ensemble strongly depends on the problem in question; see Section 8 for a discussion of the required number of trajectories. Although the number of necessary trajectories may be large for any generic observable, there are usually many observables of interest for which importance sampling can be applied in a rather straightforward fashion to obtain accurate predictions with a much smaller number of trajectories. The MCWF method may offer computational advantages compared to the direct evolution of the density matrix. First and foremost, the trajectories are completely independent of each other and their computation is embarrassingly parallel. Second, the problem size for computing any individual trajectory scales only linearly and only with the number of spatial grid points, but not with the internal quantum numbers. Although the number of angular momentum or internal quantum states influences the necessary number of trajectories, importance sampling techniques or physically sensible restrictions on the simulation parameters could have a strong limiting effect. Lastly, the MCWF method does not require an artificial truncation of the angular momentum or internal quantum numbers.

4.2.1. The waiting time approach

The main drawback of the naive implementation of the MCWF evolution given in Eq. (6) is that jumps take a finite amount of time dt equal to one step in the time discretization of the deterministic evolution with H_{eff}. As such, the time between jumps is systematically underestimated. We use instead the waiting time approach (see [43] and references therein) in the numerical implementation of the MCWF method, since it treats the quantum jumps as instantaneous. We always consider a normalized, pure initial state $|\psi(t_i)\rangle$ in the waiting time approach. The probability $p_{\text{Jump}}(t_{i+1}, t_{i+1} + \delta t)$ that the next consecutive jump occurs in the interval $[t_{i+1}, t_{i+1} + \delta t]$ is given by the difference $p_{\text{NoJump}}(t_{i+1}) -$

[3] dt can always be chosen small enough such that $\Gamma(t)dt \leq 1$.

$p_{\text{NoJump}}(t_{i+1}+\delta t)$ where $p_{\text{NoJump}}(t)$ is the probability of no jump occurring until time t. This can be linearized in terms of the jump rate $R_{\text{Jump}}(t_{i+1})$ as

$$\begin{aligned}
&\int_{t_{i+1}}^{t_{i+1}+\delta t} dt' R_{\text{Jump}}(t') \\
&= R_{\text{Jump}}(t_{i+1})\,\delta t + \mathcal{O}(\delta t^2) \\
&= p_{\text{Jump}}(t_{i+1}, t_{i+1}+\delta t) = p_{\text{NoJump}}(t_{i+1}) - p_{\text{NoJump}}(t_{i+1}+\delta t) \\
&= -\frac{dp_{\text{NoJump}}(t_{i+1})}{dt_{i+1}}\,\delta t + \mathcal{O}(\delta t^2)\,. \qquad (8)
\end{aligned}$$

The probability that no jump has occurred and only deterministic, non-unitary evolution with H_{eff} has taken place from time t_i to t_{i+1} is given by the norm of the state evolved just with H_{eff} to time t_{i+1}

$$\begin{aligned}
&p_{\text{NoJump}}(t_{i+1}) = ||\psi(t_{i+1})\rangle|^2 \\
&= \langle\psi(t_i)|\, e^{+i\int_{t_i}^{t_{i+1}} dt' H_{\text{eff}}^\dagger} e^{-i\int_{t_i}^{t_{i+1}} dt' H_{\text{eff}}}\, |\psi(t_i)\rangle\,, \\
&|\psi(t_{i+1})\rangle \equiv e^{-i\int_{t_i}^{t_{i+1}} dt' H_{\text{eff}}(t')}\, |\psi(t_i)\rangle\,. \qquad (9)
\end{aligned}$$

We obtain the jump rate $R_{\text{Jump}}(t_{i+1})$ at time t_{i+1} as the derivative of the norm. This can be related to the expectation value of the width Γ for the state at time t_{i+1} by taking the derivative of Eq. (9),

$$\begin{aligned}
R_{\text{Jump}}(t_{i+1}) &= -\frac{dp_{\text{NoJump}}(t_{i+1})}{dt_{i+1}} = -\frac{d\,||\psi(t_{i+1})\rangle|^2}{dt_{i+1}} \\
&= \langle\psi(t_{i+1})|\,\Gamma\,|\psi(t_{i+1})\rangle = \Gamma(t_{i+1}). \qquad (10)
\end{aligned}$$

The instantaneity of the jumps is the main advantage of the waiting time approach, i.e. the time interval size δt that is connected to the jumps can be chosen independently from the time interval dt that is related to the evolution with H_{eff} and thus determines the numerical error due to the finite time discretization.

A simple way to obtain the distribution in Eq. (10) is to generate one uniformly distributed random number $r_i \in [0,1]$ for each t_i and evolve with H_{eff} until $|\psi(t_{i+1})|^2 \le r_i$. At this time, a jump occurs, and additionally, one or more random numbers $r_i' \in [0,1]$ may be required to determine which collapse operator C_n with probability

$$p_n = \frac{\langle\psi(t_{i+1})|\,C_n^\dagger C_n\,|\psi(t_{i+1})\rangle}{\sum_n \langle\psi(t_{i+1})|\,C_n^\dagger C_n\,|\psi(t_{i+1})\rangle}\,, \qquad (11)$$

is applied in the jump. The jumped state at time t_{i+1} is normalized again and becomes the new initial state,

$$|\psi(t_{i+1})\rangle = \frac{C_n\,|\psi(t_{i+1})\rangle}{\big|\big|C_n\,|\psi(t_{i+1})\rangle\,\big|\big|}\,. \qquad (12)$$

A new random number $r_{i+1} \in [0,1]$ is drawn and the cycle restarts with $|\psi(t_{i+1})\rangle$ as the initial state deterministically time evolved with H_{eff} until a jump is triggered or the simulation end time is reached.

4.2.2. Algorithm for deterministic evolution between quantum jumps

For the deterministic evolution of the wave-function between jumps, we use a split-step pseudo-spectral or Strang splitting method [45,61,62]. Due to the radial symmetry of the underlying potentials, one may compute the deterministic evolution of a state with any given angular momentum l and internal (color c) quantum number using

$$u(r, t+dt) = \exp(-iH_{\text{eff}}dt)u(r,t)\,, \qquad (13)$$

where $u(r,t) = rR(r,t)$ with $R(r,t)$ being the radial part of the wave-function. The effective Hamiltonian H_{eff} as defined in Eq. (3) contains the angular momentum term and a potential $V(r)$ which depends on the internal quantum number(s). For the normalization of the wave-function, we take

$$\frac{\mathtt{L}}{\mathtt{NUM}} \sum_{r=1}^{\mathtt{NUM}} u^*(r,t)u(r,t) = 1\,. \qquad (14)$$

We enforce the boundary condition $u(r=0,t)=0$ through a real-valued Fourier sine series in a domain $r \in (0, \mathtt{L}]$ to describe both the real and imaginary parts of the wave-function. To perform the update specified in Eq. (13), we split the Hamilton operator into its kinetic and effective potential parts $H_{\text{eff}} = T_{\text{kin}} + V_{\text{eff}}$; the entire dependence on angular momentum l and internal (color c) quantum numbers and the non-hermiticity are in the effective potential term V_{eff}. We use the Baker-Campbell-Hausdorff theorem to approximate

$$\begin{aligned}
\exp[-iH_{\text{eff}}dt] \simeq \exp\left[-iV_{\text{eff}}\frac{dt}{2}\right] \exp[-iT_{\text{kin}}dt] \exp\left[-iV_{\text{eff}}\frac{dt}{2}\right] \\
+ \mathcal{O}(dt^2)\,. \qquad (15)
\end{aligned}$$

The resulting sequence for a single time step of deterministic evolution is:

1. Update in configuration space using a half-step: $\psi_1 = \exp\left[-iV_{\text{eff}}\frac{dt}{2}\right]\psi_0$.
2. Separately Fourier sine transform real and imaginary parts: $\tilde{\psi}_1 = \mathbb{F}_s[\Re\psi_1] + i\mathbb{F}_s[\Im\psi_1]$.
3. Update in momentum space using: $\tilde{\psi}_2 = \exp[-iT_{\text{kin}}dt]\,\tilde{\psi}_1$.
4. Separately inverse Fourier sine transform real and imaginary parts: $\psi_2 = \mathbb{F}_s^{-1}[\Re\tilde{\psi}_2] + i\mathbb{F}_s^{-1}[\Im\tilde{\psi}_2]$.
5. Update in configuration space using a half-step: $\psi_3 = \exp\left[-iV_{\text{eff}}\frac{dt}{2}\right]\psi_2$.

For real valued potentials, the Strang splitting leads to manifestly unitary evolution. In order to evaluate operators in momentum space, we use the Fourier sine transform ($\mathbb{F}_s$) and inverse Fourier sine transform ($\mathbb{F}_s^{-1}$) provided by the FFTW3 package [63]. The default implementation of the momentum-space operator is $T_{\text{kin}} = p^2/m$; however, there is an alternative implementation of the second derivative via centered discrete differences, $T_{\text{discrete}} = 2[1-\cos(p\,dr)]/(m\,dr^2)$ (see Sec. 3.2).

We note that a major benefit of the DST algorithm is that when using the full derivative operator T_{kin} the derivatives are effectively computed using all points on the lattice rather than a fixed subset of points. As a result, the evolution obtained using the DST algorithm is more accurate than that obtained using, for example, a Crank-Nicolson (CN) scheme with a three-point second-derivative [45]. Using the same sized derivative stencil as the default DST algorithm, the CN scheme scales as $\mathcal{O}(N^2)$ where N is the number of lattice points in the wave-function, whereas the split-step DST evolution scales as $\mathcal{O}(N\log N)$ [45].

4.3. In-medium quarkonium evolution in OQS+pNRQCD

In the following we review the ingredients of the master equation obtained from OQS+pNRQCD [24,25] at order $\mathcal{O}(r^2)$ and leading order in E/T, i.e. for compact, Coulombic quarkonia in the regime $1/a_0 \gg T, m_D \gg E$, for which the master equation has Lindblad form.

4.3.1. Hamiltonian in OQS+pNRQCD

In this regime, the Hamiltonian at order $\mathcal{O}(r^2)$ is given as

$$H = \begin{pmatrix} h_s & 0 \\ 0 & h_o \end{pmatrix} + \frac{r^2}{2}\gamma \begin{pmatrix} 1 & 0 \\ 0 & \frac{N_c^2-2}{2(N_c^2-1)} \end{pmatrix}, \tag{16}$$

where $h_{s,o}$ is the singlet, octet in-vacuum Hamiltonian in pNRQCD [53,54]:

$$h_{s,o} = \frac{\mathbf{p}^2}{m} + V_{s,o}(r)\,. \tag{17}$$

Since we assume that the quarkonium is a Coulombic bound state, the potential $V_{s,o}(r)$ in Eq. (17) is the attractive, singlet Coulomb potential $V_s(r) = -C_F\,\alpha_s/r$ where $C_F = (N_c^2-1)/(2N_c)$ or the repulsive, octet Coulomb potential $V_o(r) = +\alpha_s/(2N_c r)$. In writing Eq. (16), we have assumed a component structure of the quarkonium state as $|\psi\rangle = (|S\rangle, |O\rangle)^T$.

The coefficient γ of the order $\mathcal{O}(r^2)$ term in Eq. (16) is the dispersive counterpart of the temperature dependent heavy-quark momentum diffusion coefficient κ. It enters by matching pNRQCD with QCD.

4.3.2. Collapse operators, partial widths, and color state transitions in OQS+pNRQCD

There are six collapse operators: C_i^0 and C_i^1 (the spatial index i corresponds to the spatial directions and assumes values 1,2,3). The C_i^0 induce transitions between singlet and octet states, while the C_i^1 induce transitions among octet states. They read

$$C_i^0 = \sqrt{\frac{\kappa}{N_c^2-1}}\, r^i \begin{pmatrix} 0 & 1 \\ \sqrt{N_c^2-1} & 0 \end{pmatrix},$$
$$C_i^1 = \sqrt{\frac{(N_c^2-4)\kappa}{2(N_c^2-1)}}\, r^i \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}. \tag{18}$$

The coefficient κ is the heavy-quark momentum diffusion coefficient [64,65]; like its dispersive counterpart γ, it enters by matching pNRQCD with QCD.

Two partial decay widths (cf. Eq. (2)) are defined through the collapse operators

$$\Gamma^0 = \sum_i \kappa r^i \begin{pmatrix} 1 & 0 \\ 0 & \frac{1}{N_c^2-1} \end{pmatrix} r^i,$$
$$\Gamma^1 = \sum_i \frac{\kappa(N_c^2-4)}{2(N_c^2-1)} r^i \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} r^i. \tag{19}$$

Their sum gives the total decay width,

$$\Gamma = \sum_i \kappa r^i \begin{pmatrix} 1 & 0 \\ 0 & \frac{N_c^2-2}{2(N_c^2-1)} \end{pmatrix} r^i. \tag{20}$$

The transition probabilities are more easily obtained in the color than in the angular momentum sector. Singlet states can only jump to octet states: $p_{s\to o} = 1$. Octet states jump to octet states with probability $p_{o\to o} = (N_c^2-4)/(N_c^2-2)$[4] and to singlet states with probability $p_{o\to s} = 2/(N_c^2-2)$.

[4] This ratio is obtained by comparing the expectation value of the partial width Γ^1 to the that of the total width for any octet state.

4.3.3. Structure of the density matrix

Let us examine the general structure of the density matrix ρ for any operator $H_{\rm eff}$ that is diagonal (i.e. depends only on $r = \sqrt{\boldsymbol{r}^2}$) in the angular momentum quantum number l and all internal quantum numbers c (i.e. color in OQS+pNRQCD). The radial and angular parts of the wave-functions can be separated multiplicatively; the latter is given in terms of spherical harmonics. Any density matrix can be expanded on the sphere in terms of spherical harmonics as

$$\rho_{lm;l'm'} \propto \int d\Omega(\hat{r})\, d\Omega(\hat{r}')\, Y_{lm}(\hat{r})\, \rho\, Y^*_{l'm'}(\hat{r}')\,. \tag{21}$$

Since there is azimuthal symmetry, the magnetic quantum number m can always be summed over, and we can define a reduced density matrix $\rho_{l;l'}$ that is independent of m. Since the effective Hamiltonian is diagonal in angular momentum and internal quantum numbers, the components of the density matrix are characterized as $\rho = (\rho_{cl}(r))$, where c indicates the internal (color) quantum number and $l = 0, 1, 2, \ldots$ indicates the angular momentum quantum number. We understand $\mathrm{Tr}\,[\rho_{cl}(r)]$ as the probability to find the subsystem in any state with angular momentum squared equal to $l(l+1)$ and internal quantum number c. The total density matrix ρ of the subsystem can be cast into the block-diagonal form

$$\rho = \begin{pmatrix} \rho_{s0} & 0 & 0 & 0 & 0 & 0 \\ 0 & \rho_{s1} & 0 & 0 & 0 & 0 \\ 0 & 0 & \ddots & 0 & 0 & 0 \\ 0 & 0 & 0 & \rho_{o0} & 0 & 0 \\ 0 & 0 & 0 & 0 & \rho_{o1} & 0 \\ 0 & 0 & 0 & 0 & 0 & \ddots \end{pmatrix}, \tag{22}$$

which is preserved at all times if it has been realized at any time t_0, i.e. in the initial state. For simplicity of the following discussions, let us assume that we have a density matrix of states with a well-defined angular momentum and a well-defined internal quantum number, i.e. only a single non-vanishing block of the density matrix. Since the Lindblad equation is linear in ρ, it is trivial to generalize these considerations.

4.3.4. Angular momentum transitions

In order to understand angular momentum transitions, we need to remember that the jump term is $\sum_n C_n \rho C_n^\dagger$, with C_n defined in Eq. (18), where $n = (c, i)$ is a multi-index that ranges over $i = 1, 2, 3$ for its spatial part and $c = 0, 1$ for its internal (color) part. Hence, the term that contributes in the master equation involves a sum over all spatial components, with two powers of the same collapse operator; this results in a radially symmetric function of r^2, and preserves the block-diagonal structure of the density matrix in Eq. (22). Any individual jump operator C_n induces a (chromoelectric) dipole transition: namely, when applied to any state with well-defined angular momentum quantum number l, C_n can produce only a state with angular momentum quantum number $l' = l \pm 1$. For either internal quantum number component of the collapse operators (either value of $c = 0, 1$), we have the probability to jump to either of the (up to two) values of l' given by

$$p(l \to l') = \frac{\sum_i \left|\langle \psi_{c'l'}(r)|\, C_i^c\, |\psi_{cl}(r)\rangle\right|^2}{\sum_{i'} \sum_{l''=l\pm1} \left|\langle \psi_{c'l''}(r)|\, C_{i'}^c\, |\psi_{cl}(r)\rangle\right|^2}$$
$$= \frac{\sum_i \sum_{m,m'} \left|Y^*_{l'm'} r_i Y_{lm}\right|^2}{\sum_{i'} \sum_{l''=l\pm1} \sum_{m,m''} \left|Y^*_{l''m''} r_{i'} Y_{lm}\right|^2}$$

$$= \frac{1}{(2l+1)} \sum_i \sum_{m,m'} \left| Y^*_{l'm'} \frac{r_i}{r} Y_{lm} \right|^2 , \tag{23}$$

where the prefactor is due to normalization of spherical harmonics. For each l, the probabilities to jump to higher or lower l are

$$p_{l,\pm} = p(l \to l \pm 1) = \frac{1}{2l+1} \sum_{m,m'} \left| Y^*_{l\pm1\, m'} \left(\frac{r_i}{r}\right) Y_{lm} \right|^2 , \tag{24}$$

respectively, with $p_{l,+} = 1 - p_{l,-}$. We generally have $p_{l,-} = \frac{l}{2l+1} < p_{l,+} = \frac{l+1}{2l+1}$, which is straightforward to obtain from recurrence relations. As jumping to higher angular momentum is always favored, equilibration is not reached before $l = \infty$. This is not surprising, as the evolution in the set of $l = 0, 1, 2, \ldots$ must be a directed random walk away from the lower bound.

4.3.5. Structure of the jump probability matrix

Combining both observations, we can sketch the structure of the jump probability matrix as

$$p_{jump} = \begin{pmatrix} 0 & \frac{2}{N_c^2-2} p_{l\to l'} \\ p_{l\to l'} & \frac{N_c^2-4}{N_c^2-2} p_{l\to l'} \end{pmatrix} ,$$

$$p_{l\to l'} = \begin{pmatrix} 0 & \frac{1}{3} & 0 & 0 & \ldots \\ 1 & 0 & \frac{2}{5} & 0 & \ldots \\ 0 & \frac{2}{3} & 0 & \frac{3}{7} & 0 \\ 0 & 0 & \frac{3}{5} & 0 & \ddots \\ \vdots & \vdots & 0 & \ddots & \ddots \end{pmatrix} . \tag{25}$$

The angular momentum probability matrix $p_{l\to l'}$ is infinitely large. We generate independent random numbers for the angular momentum and the color transitions.

Note that there are no jump matrices for the particular phenomenological potentials of Appendix E.2 and Appendix E.3 since these interactions do not permit transitions that are off-diagonal in angular momentum or internal quantum numbers. However, the preceding considerations can be straightforwardly extended to arbitrary forms of interactions with jump matrices that lead to radially symmetric partial widths.

4.3.6. Hydrodynamic medium evolution with time and temperature dependent potential

The entirety of the in-medium effects in OQS+pNRQCD in the regime $1/a_0 \gg T, m_D \gg E$ at order $\mathcal{O}(r^2)$ in the multipole expansion and leading order in E/T is contained in the coefficients κ and γ. Changes in the hot, nuclear medium during the lifetime of the QGP induce changes in κ and γ. For a locally equilibrated medium such that a temperature can be defined (locally), the full temperature dependence of the master equation follows from these two coefficients; on dimensional grounds, they are expected to scale as $\kappa \equiv \hat{\kappa} T^3$ and $\gamma \equiv \hat{\gamma} T^3$, where $\hat{\kappa}$ and $\hat{\gamma}$ are dimensionless numbers that may have further, nontrivial temperature dependence. Details of the time evolution of the medium and its temperature rely on a hydrodynamical description of the medium; for simplified benchmarks and code tests, Bjorken evolution can be used, i.e. $T(\tau) = T_0(\tau_{\rm med}/\tau)^{1/3}$ (see Table 1). Results for $\kappa(T)$ and $\gamma(T)$ have been obtained from lattice gauge theory simulations in local thermal equilibrium [66,67]; however, some modeling of their temperature dependence constrained by the lattice results is still necessary. Such lattice based models coupled to a hydrodynamical evolution of the medium could, therefore, describe how these heavy-quark momentum diffusion coefficients evolve in time.

4.4. Summary of `QTraj` execution

We collect the relevant information and enumerate a practical algorithm taking the form of a multi-loop structure.

1. The outermost loop (scripts external to the actual `QTraj` code) is over possible physics scenarios. This includes varying `Grid`, `Physics`, `Potential`, and `Temperature parameters`, e.g. Bjorken evolution or different hydrodynamical evolutions encoded by different "temperature evolution" or "trajectory files" (see Appendix C). The final time of each trajectory's evolution is determined by the time at which the temperature along the trajectory drops below T_f after which no further evolution is necessary since below T_f the vacuum potential is used.
2. The next loop (still external to the `QTraj` code) is over initial states of the subsystem. Its density matrix is a linear combination of pure states $|\psi_{cl}(0)\rangle$ with well-defined angular momentum (l) and internal (c) quantum numbers
$$\rho(0) = \sum_{\psi_{cl}} p_{\psi_{cl}} |\psi_{cl}(0)\rangle \langle\psi_{cl}(0)| . \tag{26}$$
We select from among the states $|\psi_{cl}(0)\rangle$ with the appropriate *initial state probability* $p_{\psi_{cl}}$. In the current implementation, the initial state is typically a vacuum eigenstate or a Gaussian multiplied by an appropriate power of r to realize the desired angular momentum in the initial state; such initial states can be color singlet or octet.
3. The actual `QTraj` simulation code begins here. We first compute one quantum trajectory with in-vacuum evolution of $|\psi_{cl}(0)\rangle$ until the onset of coupling to the medium at t_0; the result $|\psi_{cl}(t_0)\rangle$ is stored and reused later. For each quantum trajectory, we define $|\psi_{cl}(t_i)\rangle \equiv |\psi_{cl}(t_0)\rangle$ for $i = 0$. The zeroth trajectory completes without jumps, and its final state overlaps are recorded. The corresponding final state norm equals the probability $p_{\rm NoJumps}$ of no jumps occurring over the entire course of the evolution. The outermost loop of `QTraj` is over a set of `nTrajectories` quantum trajectories for the given initial condition.
4. Each quantum trajectory is computed in a separate loop over the evolution time, starting with the current state of the random number generator (as after the previous quantum trajectory); the code alternates between intermittent, iterative $H_{\rm eff}$ evolution and instantaneous jumps for each individual quantum trajectory.
   (a) We draw a random number $r_i \in [0, 1]$; in the first step of the first trajectory $i = 0$, we distinguish two cases:
      i. If $r_0 <$ `rMin` $\equiv p_{\rm NoJumps}$, we terminate the quantum trajectory and default to the known no-jump final state computed in step 3.
      ii. If $r_0 >$ `rMax` (see Table 1) we terminate the quantum trajectory and report total loss for all low-lying bound states (zero overlaps).

   In the innermost loop, we evolve the state $|\psi_{cl}(t_i)\rangle$ with $H_{\rm eff}$ to time t_{i+1} while $r_i < ||\psi_{cl}(t_{i+1})\rangle|^2$ and $t_{i+1} < t_{\rm final}$.
   (b) If $r_i \geq ||\psi_{cl}(t_{i+1})\rangle|^2$ and $t_{i+1} \leq t_{\rm final}$, we determine if a jump occurs. If the simulation parameter `maxJumps` (see Table 1) is larger than the number of previous jumps `nJumps`, i.e. `nJumps` < `maxJumps`, we perform a jump. Otherwise, in the context of OQS+pNRQCD, we conclude that the subsystem has evolved too far away from the low-lying in-vacuum states to return in a finite amount of time; we thus terminate the quantum trajectory and report total loss for all low-lying bound states (zero overlaps). In the OQS+pNRQCD framework, a jump as defined by the

collapse operators in Eq. (18) entails a change of angular momentum, a possible change of color state, and a multiplication of the radial wave-function by r.

i. For $l = 0$, the state jumps to $l = 1$ with probability 1. For $l > 0$, we draw a random number $r_{il} \in [0, 1]$ to determine whether to jump up or down in the angular momentum quantum number. We denote the jumped value of the angular momentum quantum number l' and the trajectory $|\psi_{cl'}(t_{i+1})\rangle$.

ii. For $c = s$, i.e. a singlet state, a jump to an octet state occurs with probability 1. For $c = o$, i.e. an octet state, we draw an additional random number $r_{ic} \in [0, 1]$ to determine whether to jump in the internal quantum number from octet to octet or octet to singlet. We denote the jumped value of the internal quantum number c' and the trajectory $|\psi_{c'l'}(t_{i+1})\rangle$.

iii. The final form of the jumped state is obtained by multiplying $|\psi_{c'l'}(t_{i+1})\rangle$ by the radial part of the jump operator and normalizing, i.e. $\left|\tilde{\psi}_{c'l'}(t_{i+1})\right\rangle = r\,|\psi_{cl}(t_{i+1})\rangle\,/||r\,|\psi_{cl}(t_{i+1})\rangle\,||$; the change in angular momentum and/or internal quantum number is implemented in steps 4(b)i and 4(b)ii.

We proceed to the $(i + 1)$ th iteration of the cycle for this quantum trajectory, i.e. generate a new random number $r_i \in [0, 1]$ and repeat step 4b with $\left|\tilde{\psi}_{c'l'}(t_{i+1})\right\rangle$. Note that in the subsequent evolution with H_{eff} the angular momentum and internal quantum number dependent terms, i.e. the singlet or octet potentials $V_{s,o}(r)$ as defined below Eq. (17), are realized using the jumped values of the quantum numbers in the subsequent evolution, i.e. l' and c'.

(c) Each set of final state overlap ratios or overlaps together with metadata is written to disk as specified in Appendix D.

5. The final state overlaps from all quantum trajectories are written to `output/ratios.tsv`. We describe how to process and visualize the results in Section 6.

Note that the evolution with a phenomenological potential is performed in the same way, except that there are no jumps; hence, without jumps, step 4 in its entirety in the summary of the execution does not exist.

5. Key parts of the code

All C source code for the `QTraj` package can be found in the `build/src` and `build/include` directories. The `.cpp` files contained in `build/src` are

```
build/src
├─ eigensolver_arma.cpp
├─ eigensolver_mkl.cpp
├─ initialcondition.cpp
├─ interpolator.cpp
├─ outputroutines.cpp
├─ paramreader.cpp
├─ potential.cpp
├─ qtraj.cpp
├─ trajectory.cpp
└─ wavefunction.cpp
```

The main entry point is `build/src/qtraj.cpp`. The main routine found in this file calls routines defined in the other files. In the following subsections, we present a number of key parts of the code to assist users in understanding the core subroutines which evolve the wave-function between quantum jumps and the central logic used to implement stochastically sampled quantum jumps. Note that lines that begin with "." indicate portions of the code that have been omitted for readability.

5.1. Split-step pseudo-spectral wave-function update

Between quantum jumps, `QTraj` uses a split-step pseudo-spectral method to evolve the wave-function as described in Sec. 4.2.2. This is implemented by the routine `makeStep` shown below and defined in the file `wavefunction.cpp`. This routine calls the `DST` function which performs the discrete sine transform of the real and imaginary parts of the wavefunction separately and then sets the double complex wave-function variable to the required DST (in-place transform). The variables `in` and `out` are buffers used to hold intermediate results. The variable `spaceKernel` contains $\exp(-idt\,V_{\text{eff}}/2)$, which are the values of the spatial part of the time evolution operator evaluated at each point on the spatial lattice. Note that as the potential is time dependent the variable `spaceKernel` must be updated at every time step. The variable `momKernel` contains $\exp(-idt\,T_{\text{kin}})$ which are the values of the momentum-part of the time evolution operator evaluated at each point on the conjugate momentum lattice. Note that the kinetic energy operator is time-independent, and, hence, `momKernel` only needs to be loaded during the initialization stage.

```
void makeStep(fftw_plan p, dcomp* wfnc, double* in,
     double* out, dcomp* spaceKernel, dcomp*
     momKernel)
{
    // make one spatial half-step
    for (int i=0; i<num; i++) wfnc[i] = spaceKernel[
     i]*wfnc[i];
    // forward DST transform
    DST(p, wfnc, in, out);
    // make one full step in momentum space
    for (int i=0; i<num; i++) wfnc[i] = momKernel[i
     ]*wfnc[i];
    // backward DST transform; it's its own inverse
    DST(p, wfnc, in, out);
    // make one spatial half-step
    for (int i=0; i<num; i++) wfnc[i] = spaceKernel[
     i]*wfnc[i];
}
```

5.2. Wave-function evolution without jumps

The `QTraj` code includes the capability to run with and without quantum jumps. If jumps are turned off, then `QTraj` simply evolves the wave-function using the complex effective Hamiltonian H_{eff}. This is implemented in the routine `evolveWavefunction` listed below. This routine takes two arguments: `nStart` which is the integer index of the starting time for the evolution and `nSteps` which is the number of time steps to evolve the wave-function forward in time.

```
double evolveWavefunction(int nStart, int nSteps) {

    double norm = 1;
    double rexp;

    // begin time loop
    for (int n=nStart;n<nStart+nSteps;n++) {

        // update the space kernel
        loadSpaceKernel(T[n]);

        // output info to screen and/or disk
        if (n%snapFreq==0 || n==nStart) {
```

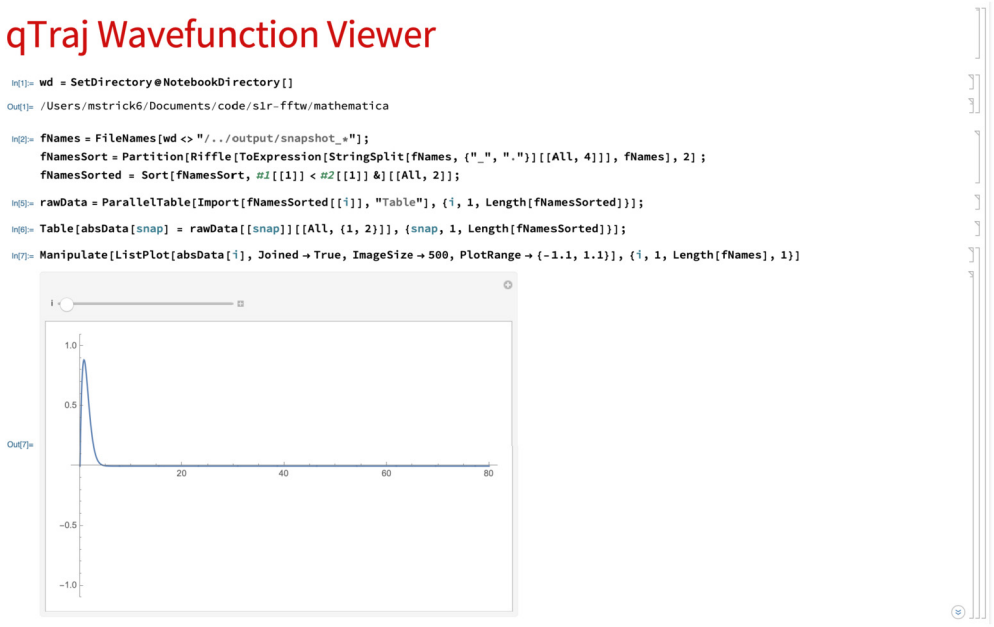


Fig. 1. Example `viewer.nb` output.

```
            norm = computeNorm(wfnc);
            outputInfo(n,norm,false);
        }

        // update the wavefunction
        makeStep(p,wfnc,in,out,spaceKernel,momKernel
    );

    } // end time loop

    // output final wavefunction, summary info, and
     ratios
    norm = computeNorm(wfnc);
    outputInfo(nStart+nSteps,norm,false);
    return norm;

}
```

5.3. Wave-function evolution with jumps

A compactified listing of the `evolveWavefunctionWithJumps` routine found in the file `trajectory.cpp` is shown below (see also Section 4.4). The first argument `nStart` is the index of the starting time for the evolution and the second argument `rMin` is the minimum initial random number to consider. This variable is computed by first running a complete H_{eff} evolution to determine the final norm. As any random number smaller than the final H_{eff} norm results in no quantum jumps, there is no reason to check for jumps. In this case, the routine sets the overlaps to those determined from the H_{eff} evolution performed during initialization.

In addition, as can be seen from the code listing below, the global parameter `rMax` sets the maximum initial random number accepted by the algorithm. If the initial random number is less than `rMax`, `QTraj` assumes that the stochastic evolution will result in too many jumps for a significant final overlap with singlet bound states. The default for this parameter is 1; reducing from 1 results in faster code execution. This faster execution, however, results in a systematic uncertainty (see Fig. 9 and surrounding discussion).

After this initial check, the evolution proceeds by generating a random number uniformly distributed between 0 and 1, then evolving the wave-function until the norm squared of the wave-function drops below this number. At this point, a quantum jump is triggered and two additional random numbers are generated to determine the outgoing color and angular momentum quantum numbers. Finally, the wave-function is multiplied by r and re-normalized using the `doJump` subroutine. The routine then continues the time loop by generating a new random number and evolving the wave-function forward in time with the updated quantum numbers. In all cases, the final result from this routine is stored in a double array called `singletOverlaps`.

```
void evolveWavefunctionWithJumps(int nStart, double
     rmin) {
    .
    .
    // generate initial random number
    double r = Random();
    firstRandomNumber = r; // save this for testing
     output
    // check to see if jumps will be triggered at
     all
    if (r<rmin) {
        print_line();
        cout << "==> No jumps necessary.  ";
        cout << "Setting overlaps to Heff overlaps."
      << endl;
        retrieveNoJumpsSingletOverlaps();
        return;
    }
    // check to see if upper threshold is crossed
    if (r > rMax) {
      print_line();
        cout << "==> Initial random number greater
     than rmax.  ";
        cout << "Setting overlaps to zero." << endl;
        for (int i=0; i<nBasis; i++) singletOverlaps
     [i] = 0;
        return;
    }
    .
    .
    // normal evolution until norm drops below
     random threshold r = Random(0,1)
    .
    .
    // do the jump if necessary
    if (n<maxSteps) {

        // output some info just before the jump
```

```
        norm = computeNorm(wfnc);
        outputInfo(n,norm,true);

        // determine new angular momentum state
        if (Random() < AngMomProb(lval)) lval -= 1;
        else lval += 1;

        // determine new color state
        if (cstate == OCTET  && Random() < 2./7.)
    cstate = SINGLET;
        else cstate = OCTET;

        // apply jump operator and normalize to 1
        doJump(wfnc);

        // generate new random # and increment
    nJumps
        r = Random();
        nJumps++;

        // output some info just after the jump
        norm = 1; // forced to be one by doJump so
    don't waste time computing it
        outputInfo(n,norm,true);
    }

    // check to see if we exceeded the maximum
     number of jumps
    if (nJumps>=maxJumps) {
        print_line();
        cout << "==> Exceeded max jumps.
     Terminating evolution." << endl;
        print_line();
        break;
    }
    .
    .
    .
}
```

6. Data analysis notebooks

`QTraj` can be set to output snapshots of the evolution of the wave-function in the directory `output` or `output-<seed>` depending on the value of `dirnameWithSeed` (see Sec. 3). The format for these files is detailed in Appendix D. The user can suppress the output of these snapshots by setting `saveWavefunctions` to 0. The only other disk output from `QTraj` is to the file `ratios.tsv` in the output directory. The format of this file depends on the `temperatureEvolution` parameter (see Sec. 3) as described in Appendix D. In order to assist the user in understanding how to process the resulting output, we include Mathematica notebooks that read all three output formats.

The `mathematica` directory in the main package contains Mathematica [68] notebooks that can be used for data analysis:

```
mathematica
├── raaCalculator.nb
├── raaCalculator-trajectories.nb
├── ratioFileReader.nb
└── viewer.nb
```

6.1. The notebook `viewer.nb`

The Mathematica notebook `viewer.nb` allows the user to visualize the wave-function evolution produced by `QTraj` . To generate the files necessary, the parameter `saveWavefunctions` must be set to 1. In this configuration, the initial wave-function, "snapshots" of the wave-function recorded every `snapFreq` steps, and the final wave-function are exported to the `output` folder as separate files named `snapshot_#.tsv` where # is the step in the evolution.

To generate output for testing, the user can execute the following example

```
$ ./qtraj -initType 0 -doJumps 0 -
saveWavefunctions 1 -dirnameWithSeed
0
```

Upon successful completion, one can evaluate all cells in the `viewer.nb` notebook. Typical output is shown in Fig. 1.

6.2. The notebook `ratioFileReader.nb`

To generate output for testing, the user can execute

```
$ ./qtraj -initType 0 -doJumps 1 -
nTrajectories 128 -dirnameWithSeed 0
```

Upon successful completion, the code generates a file `output/ratios.tsv` which contains the resulting survival probabilities (or overlaps) for the $\Upsilon(1S)$, $\Upsilon(2S)$, $\chi_b(1P)$, $\Upsilon(3S)$, $\chi_b(2P)$, and $\Upsilon_2(1D)$ states. To visualize the data, the user can open the `ratioFileReader.nb` Mathematica notebook and evaluate all cells in the notebook. The first section of the notebook produces data for the ratios averaged over all trajectories based on the output generated in `output/ratios.tsv`. A summary plot similar to Fig. 2 is generated. To add more statistics, the user can rerun the command above. The newly generated results are appended to `output/ratios.tsv`.

6.3. The notebook `raaCalculator-trajectories.nb`

Note that, although the example above runs on a single core, access to a computing facility supporting Portable Batch Scripts (PBS) enables the user to launch many `QTraj` trajectories simultaneously.[5] For this purpose, we have provided a PBS script `scripts/manyTrajectories.pbs` which automates the extraction of individual physical trajectories from a tgz file containing a large number of Monte-Carlo generated physical trajectory files individually having the format specified in Appendix C.2. The tgz file should be saved in the following location: `input/runset/-trajectories.tgz`. An example trajectories.tgz file is available in [71].

The `manyTrajectories.pbs` script has adjustable variables at the top allowing the user to set the number of cores on which to launch jobs and the number of physical and quantum trajectories to generate.

```
#PBS -l walltime=8:00:00
#PBS -l nodes=1:ppn=1
#PBS -l mem=300MB
#PBS -t 1-100
#PBS -N qtraj-trajectory
#PBS -j oe
#PBS -A PGS0253

module load fftw3

# number of physical trajectories to sample
NPTRAJ=100
```

[5] We also provide two condor scripts which can be used for launching trajectories on the Open Science Grid [69,70].

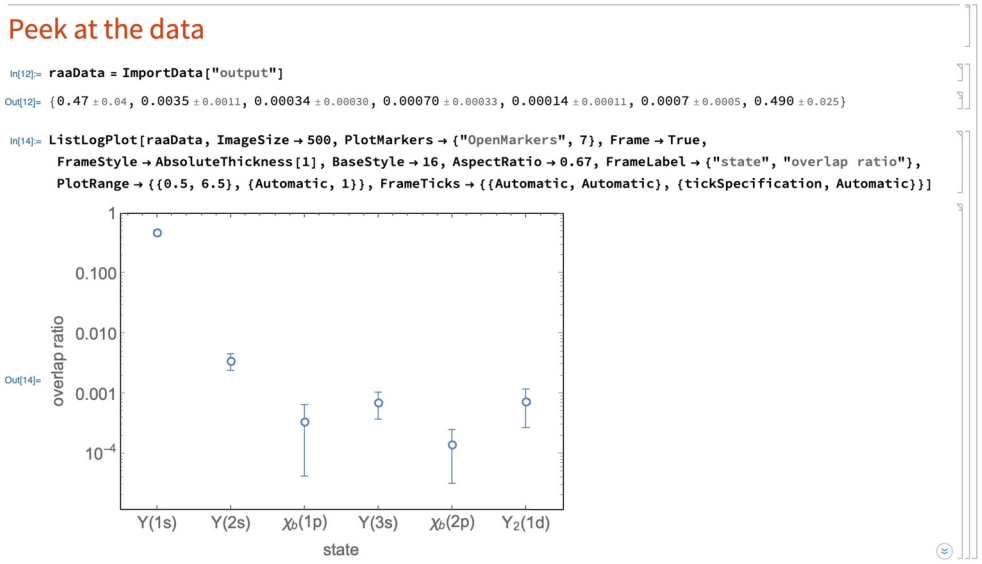


Fig. 2. Example `ratioFileReader.nb` output.

```
# number of quantum trajectories to sample per
    physical trajectory
NQTRAJ=50
  .
  .
  .
# extract random unique set of NPTRAJ trajectories
    from trajectories.tgz
tar -ztf $PBS_O_WORKDIR/input/runset/trajectories.
    tgz | shuf -n ${NPTRAJ} > trajectoryList.txt
tar -xzf $PBS_O_WORKDIR/input/runset/trajectories.
    tgz --warning=no-timestamp --files-from
    trajectoryList.txt
  .
  .
  .
for file in ./input/runset/*
do
    echo "Running with $file"
    # run the code first for l=0 then for l=1
    ./qtraj -initL 0 -nTrajectories ${NQTRAJ} -
    temperatureEvolution 2 -temperatureFile $file >
     log_0_${cnt}.txt 2> err_0_${cnt}.txt
    ./qtraj -initL 1 -nTrajectories ${NQTRAJ} -
    temperatureEvolution 2 -temperatureFile $file >
     log_1_${cnt}.txt 2> err_1_${cnt}.txt
done
  .
  .
  .
```

For post-processing, all `ratio.tsv` files produced during execution of the `manyTrajectories.pbs` script should be concatenated and combined into a single gzipped file, canonically named `datafile.gz`. This can be done using the UNIX `cat` command. This step is the responsibility of the user since different computing environments have potentially different formats for the output directory structure, etc. To process the data, the user should create a new directory in the `input` subfolder of the `mathematica` folder and place the `datafile.gz` in the new folder. An example `datafile.gz` file is supplied with the `QTraj` distribution and can be found in the `mathematica/input/example1` folder. The supplied version of `mathematica/raaCalculator-trajectories.nb` is setup to read in this example and, upon successful execution, generate a plot similar to that shown in Fig. 3.

6.4. The notebook `raaCalculator.nb`

To make runs using physical-trajectory averaged temperatures files, the `QTraj` package includes a PBS script called `scripts/scanImpactParameters.pbs`. The header and topmost variables of this script allow the user to specify the number of cores on which to run the code and the number of trajectories to generate per core.

```
#PBS -l walltime=08:00:00
#PBS -l nodes=1:ppn=1
#PBS -l mem=300mb
#PBS -t 1-100
#PBS -N qtraj-fftw
#PBS -j oe
#PBS -A PGS0253

module load fftw3

# number of quantum trajectories per impact
    parameter for each job
# total number of trajectories is the size of job
    array times this number
NTRAJ=2048

# impact parameter list
blist=("0" "2.32326" "4.24791" "6.00746" "7.77937" "
    9.21228" "10.4493"\
"11.5541" "12.5619" "13.4945" "14.3815")

  .
  .
  .
```

The example above launches `QTraj` on 100 cores, with each instance generating 2048 quantum trajectories. Once combined, this results in 204,800 quantum trajectories per point in `blist`. It is the user's responsibility to adjust the number of requested nodes using the line `PBS -t 1-100`. The range specified indicates the number of independent machines (cores) to use. The amount of time specified by `PBS -l walltime=02:00:00` is the expected maximum time for each core to complete the task. The user can set the number of quantum trajectories for each core to simulate using the variable `NTRAJ`. Finally, there is a list of impact parameters over which to loop. These should map to the in-

```
In[119]:= withFDnp = ListPlot[{raa1sNP, raa2sNP, raa3sNP}, PlotRange → {{0, 408}, Automatic}, Joined → True,
    InterpolationOrder → 1, IntervalMarkers → "Bands", FrameLabel → {"Npart", "RAA"},
    PlotLegends → Placed[LineLegend[{Style["QTraj - ϒ(1S)", 16], Style["QTraj - ϒ(2S)", 16], Style["QTraj - ϒ(3S)", 16]},
      LegendMargins → 20, LegendMarkerSize → {{49, 15}}], {Right, Top}], PlotLabel → Style[labelMap[dataDir], 18]]
```

Out[119]=

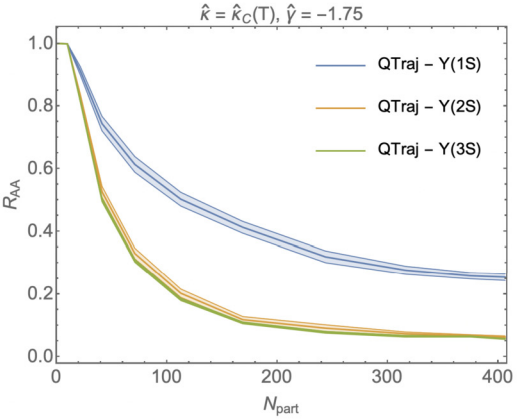


Fig. 3. Example `raaCalculator-trajectories.nb` output. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

put files of the form `input/temperature/averageTemperature_<impact parameter>.txt`.[6]

After adjusting `scripts/scanImpactParameters.pbs` to their needs, from the main directory the user can execute

```
$ qsub ./scripts/scanImpactParameters.pbs
```

to launch a job. Upon successful completion, a directory, or directories depending on the user's environment, of the form `output-<jobid>` are created. These each contain sub-folders for every impact parameter in `blist`. Inside of each impact parameter's folder, there is a file named `ratios.gz`, which is a compressed collection of `ratio.tsv` style outputs. If the user's system generates multiple output folders, the user will need to combine the ratios.gz files for each impact parameter. This can be done using the `cat` utility in UNIX, e.g., `cat output-*/0/ratios.gz >./combined/0/ratios.gz`. We provide a shell script `scripts/combine.sh` to facilitate this process.

To plot the computed survival probabilities, the user can open the Mathematica notebook `raaCalculator.nb` and adjust the paths to point to the local output directory (or the combined one created using `cat` or `scripts/combine.sh`). Upon successful execution, the user is presented with a plot similar to the one shown in Fig. 3. We note that, although the name of the notebook is `raaCalculator.nb`, this example notebook computes only the pre-feeddown survival probabilities and not R_{AA} which requires further processing by the user. We provide sample data with the `QTraj` distribution called `mathematica/input/example2`. The Mathematica notebook `raaCalculator.nb` is configured to import data from this folder as an example for the user (see Fig. 4).

7. Code performance and scaling tests

The `QTraj` distribution includes a suite of built-in self tests that can be run using `make tests`. This functionality requires that the Google testing framework be installed (see Appendix A.5). In addition, we note that apples-to-apples comparisons of `QTraj` and a QuTiP 2 implementation of the Lindblad equation solver were performed in [44]. Therein it was demonstrated that `QTraj` and QuTiP 2 implementations of the Lindblad equation solver are in agreement when the same lattice sizes, angular momentum cutoff, etc. are used.

In this section, we present the dependence of `QTraj` results on various run time parameters, e.g. lattice spacing, lattice volume, temporal step size, maximum number of jumps, etc. For all results presented in this section, we use Gaussian S-wave initial conditions with $\Delta = 0.2a_0$ and Bjorken temperature evolution starting at $\tau = 0.6$ fm with $T_0 = 425$ MeV and terminating at $T_f = 250$ MeV.

In the left panel of Fig. 5, we present the dependence of the extracted $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities on the lattice spacing with a fixed lattice volume of $L = 80$ GeV^{-1}. We obtain the survival probabilities by computing the ratio of the modulus squared of each eigenstate's overlaps with the final and initial wave-functions. We note that the square of an eigenstate overlap with the initial wave-function corresponds, in our model, to the probability of measuring that state if there were no medium. This ratio is averaged over 100,000 quantum trajectories and we report the mean along with the uncertainties obtained by computing the standard error of the mean. As can be seen from the left panel of Fig. 5, there is only a weak dependence on the lattice spacing. In order to quantify this variation, we note that the ratios of the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities obtained using 8192 and 4096 lattice points is $\{1.02 \pm 0.04, 1.11 \pm 0.08, 1.15 \pm 0.11\}$, respectively. Note that the effects of the radial discretization are naturally expected to be more severe for states with larger contributions from small radii. For past phenomenological studies, 4096 points were used with a volume of $L = 80$ GeV^{-1} [44].

In the right panel of Fig. 5, we present the dependence of the extracted $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities on the lattice volume L holding the lattice spacing fixed at $a = 5/256$ GeV^{-1}. As this figure demonstrates, for the type of initial conditions used, one finds that there is no significant volume effect for $L > 20$ GeV^{-1}. Again, in order to quantify this variation, we note that the ratios of the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities obtained using $L = 80$ and 160 GeV^{-1} are $\{1.01 \pm 0.04, 1.10 \pm 0.07, 1.16 \pm 0.10\}$, respectively. Taken together, these results demonstrate that with 100,000 trajectories the systematic uncertainties associated with finite lattice size and spacing are on the order of 5%, 10%, and 15% for the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities, respectively. Note that the effects of

[6] A set of such files is included in the standard distribution.

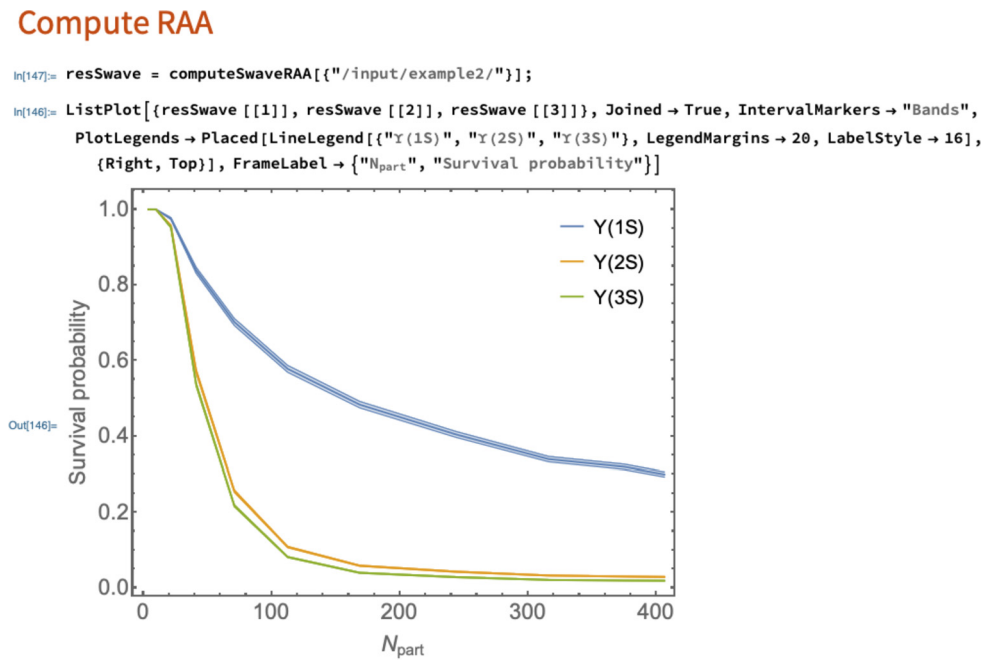


Fig. 4. Example `raaCalculator.nb` output.

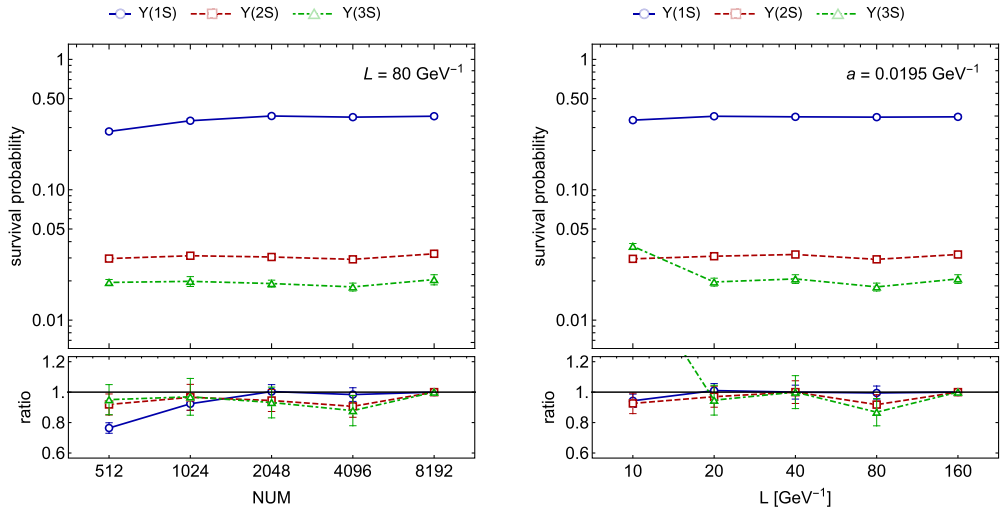


Fig. 5. (Left) Survival probabilities for $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ versus the number of lattice sites for a fixed volume $L = 80$ GeV^{-1}. (Right) Same versus the lattice volume L for fixed $a = 0.0195$GeV^{-1}. For both panels, we used Gaussian S-wave initial conditions with $\Delta = 0.2a_0$ and Bjorken temperature evolution starting at $\tau = 0.6$ fm with $T_0 = 425$ MeV. Error bars indicate the statistical uncertainty associated with averaging over 100,000 quantum trajectories.

the finite volume are naturally expected to be more severe for states with larger contributions from large radii.

Turning next to the dependence of the survival probabilities on the temporal lattice spacing Δt (`dt`), in the left panel of Fig. 6, we present the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities as a function of Δt. For each Δt, we keep $L = 80$ GeV^{-1} and `NUM` $= 4096$ and average over 100,000 quantum trajectories. As can be seen from this figure, the results show a weak dependence on the assumed temporal lattice spacing. In this case, we find that the ratios of the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities obtained using $\Delta t = 0.005$ and 0.001 GeV^{-1} are $\{1.029 \pm 0.03, 1.11 \pm 0.05, 1.15 \pm 0.08\}$, respectively. From this, we can estimate the uncertainties associated with the temporal lattice spacing to be on the order of 5%, 10%, and 15% for the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities, respectively.

In the right panel of Fig. 6, we present the dependence of the extracted 1S, 2S, and 3S survival probabilities on the initial width Δ of the Gaussian wave-function used. As can be seen from this figure, the 1S survival probability does not depend strongly on the assumed value of Δ, whereas the excited states survival probabilities only become approximately independent of Δ for $\Delta \lesssim 0.2$. In order to quantify the change, we note that the ratios of the $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ survival probabilities obtained using $\Delta/a_0 = 0.1$ and 0.02 are $\{1.00 \pm 0.04, 1.06 \pm 0.08, 1.09 \pm 0.11\}$, respectively. These are all consistent with unity within statistical errors; however, there is an indication that the corrections are larger for excited states. We note that when using $\Delta/a_0 = 0.1$ we doubled the number of quantum trajectories in order to obtain similar uncertainty estimates as $\Delta/a_0 = 0.2$ due to the fact the system jumps more frequently for smaller Δ. For this reason, for phenomenological applications, we use $\Delta/a_0 = 0.2$ since it requires substantially less computational effort.

Finally, we investigate the dependence of `QTraj` results on two optimization parameters: `maxJumps` and `rMax`. The former

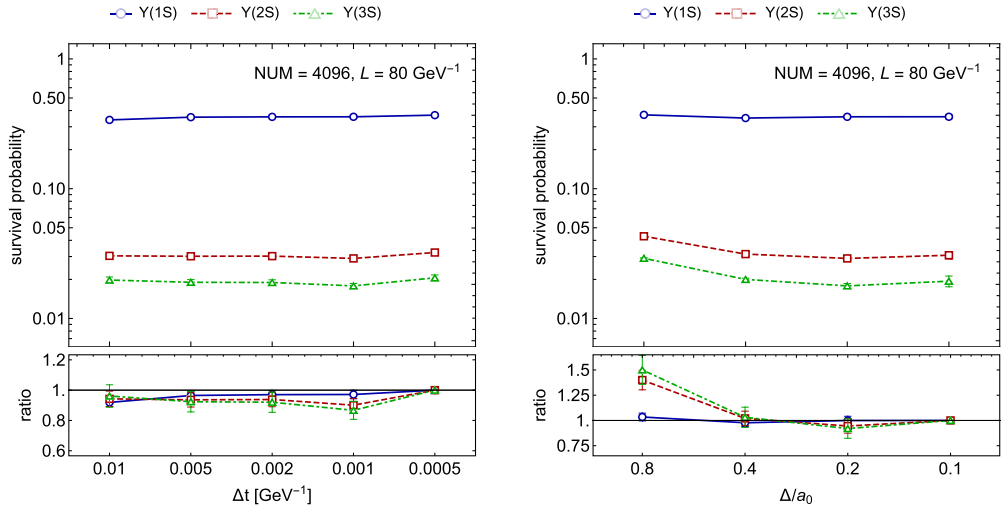


Fig. 6. (Left) Survival probabilities for $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ versus the time interval Δt for a fixed volume $L = 80\,\text{GeV}^{-1}$ and $NUM = 4096$. (Right) Same as a function of the width of the initial Gaussian Δ/a_0. For the left panel we used Gaussian S-wave initial conditions with $\Delta = 0.2a_0$. For both panels, we used Bjorken temperature evolution starting at $\tau = 0.6$ fm with $T_0 = 425$ MeV. In both panels, the error bars indicate the statistical uncertainty associated with averaging over quantum trajectories.

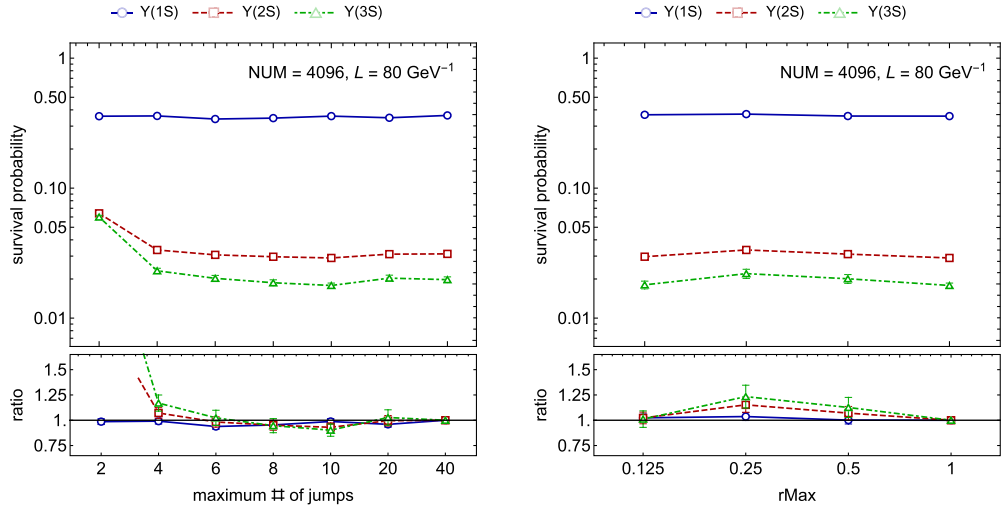


Fig. 7. (Left) Survival probabilities for $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ versus the maximum number of jumps allowed for a fixed volume $L = 80\ \text{GeV}^{-1}$ and NUM $= 4096$. (Right) Same versus `rMax`. For both panels, we used Gaussian S-wave initial conditions with $\Delta = 0.2a_0$ and Bjorken temperature evolution starting at $\tau = 0.6$ fm with $T_0 = 425$ MeV. Error bars indicate the statistical uncertainty associated with averaging over 100,000 quantum trajectories.

sets the maximum number of jumps before the state evolution is terminated and the latter sets the maximum value for the initial random number used during quantum trajectory evolution. We generate the initial random number in the range $(0, 1)$; however, if the generated initial random number is greater than `rMax`, we set the survival probability for the quantum trajectory sampled to zero. This is justified by the fact that trajectories that jump early in the quantum trajectory evolution will have essentially zero overlap with the singlet bound states of interest. In the left panel of Fig. 7, we present the 1S, 2S, and 3S survival probabilities as a function of `maxJumps`. As this panel demonstrates, the results for the $\Upsilon(1S)$ survival probability do not depend strongly on `maxJumps`, whereas the two excited states have more dependence on this parameter. Turning to the right panel of Fig. 7, we present the 1S, 2S, and 3S survival probabilities as a function of `rMax`. From this figure, we once again see that the 1S state survival probability is independent of this parameter while the excited states survival probabilities have a weak dependence on `rMax`. In practice, `rMax` < 1 can be used to accelerate code execution with the speed gain proportional to $1/$`rMax`. This functionality is useful for obtaining quick estimates of survival probabilities.

8. Code benchmarks and comparisons with existing frameworks

A key benefit of `QTraj` compared to methods relying on direct solution of the reduced density matrix evolution is the scaling of the computational complexity like `NUM` log `NUM`, the dominant contribution to which comes from the forward and inverse Fourier Sine transforms. Methods which solve the matrix evolution equation directly can be expected to scale like `NUM`$^\alpha$ with $\alpha \gtrsim 3$. Additionally, in terms of memory use, the memory footprint of `QTraj` scales like `NUM`, whereas in this respect matrix-based methods scale like `NUM`^{2}. Since, in practice, one must have large spatial grids in order to properly resolve both bound and unbound state dynamics, this places a strong limitation on such methods. We also note that since each quantum trajectory sampled by `QTraj` is statistically independent, one can parallelize the production of `QTraj` results straightforwardly.

In order to compare the two methods, we use the code developed in [24,25] which solves the same problem, but uses the

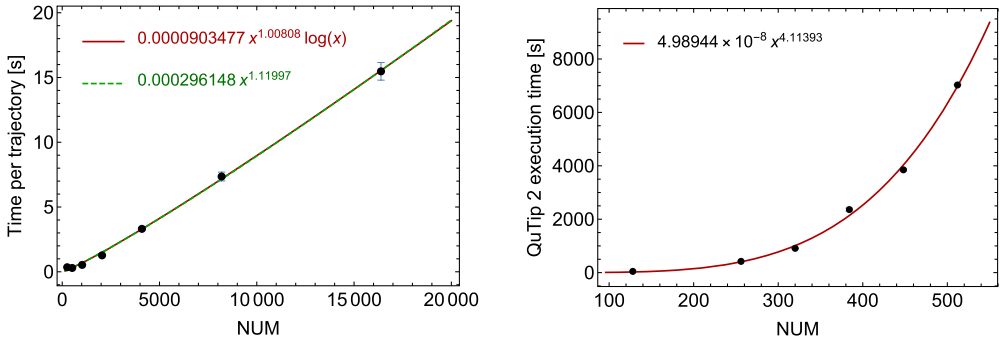


Fig. 8. (Left) Time in seconds required to compute one quantum trajectory using `QTraj` as a function of the number of discrete points in the lattice, `NUM`. Data were obtained from average using 5 runs for each `NUM` which generate 100 quantum trajectories. For the measurement, we used the UNIX utility `perf`. Solid and dashed lines show the result of two fits to these data. (Right) Time in seconds to compute the evolution of the density matrix with $l_{\max} = 1$ using QuTiP 2 as a function of the `NUM`. Data were again obtained by averaging over 5 runs for each `NUM`. Solid line shows a fit to the collected data. In both left and right panels Gaussian S-wave initial conditions were used.

mesolve function of the popular QuTiP 2 Python package for describing the dynamics of open quantum systems [24,25,37]. The same initial conditions, namely singlet S-wave Gaussian initial conditions with $\Delta = 0.2$ are considered. In the left panel of Fig. 8, we present the number of seconds required per trajectory as a function of the lattice size `NUM`. For this measurement, we use the UNIX `perf` utility and measured the mean run-time per trajectory along with the standard error of the mean. All tests were performed on an a single Intel Xeon E5-2630 v3 CPU, which runs at a clock rate of 2.40 GHz. The total memory available to the runs was 64 GB and the operating system used was Red Hat Enterprise Linux Server release 7.9.

The solid and dashed lines in Fig. 8 show the result of two fits to these data. The solid red line indicates the result obtained when assuming a fit function of the form $A\,\mathrm{NUM}^B \log \mathrm{NUM}$. We note, importantly, that the extracted coefficient B is very close to unity in this case, suggesting that the scaling of the `QTraj` execution time is dominated by the underlying FFTs which is predicted to give scaling of the form $\mathrm{NUM} \log \mathrm{NUM}$. As an alternative, the green dashed line shows results obtained using a fit form $A\,\mathrm{NUM}^B$. As one can see, in this range the two fits to the `QTraj` performance data are virtually indistinguishable and one finds that they have similar χ^2 per degree of freedom. We find that these two fits to the `QTraj` single trajectory time are within 5% of one another for $512 \leq \mathrm{NUM} \leq 2.5 \times 10^5$.

In the right panel of Fig. 8, we present run-times for the QuTiP 2 based solver [24,25,37]. In this case, one is restricted to much smaller lattices due to the memory footprint of the matrix-based solvers; $\mathrm{NUM} = 512$ is the largest lattice that we can run given the total memory of the machine used for testing (64 GB). From the right panel of Fig. 8, we see that the QuTiP 2 run-time data collected are well-described by a fit of the form $A\,\mathrm{NUM}^B$ with $B \sim 4$. We note that the measurements presented in the left and right panels of Fig. 8 are not directly comparable since the left panel presents the single trajectory run times. In order to compare the total run times, one should multiply the `QTraj` single-trajectory estimates by the number of quantum trajectories that are necessary to obtain convergence within a given bound. In practice, for the application to bottomonium suppression (R_{AA}) one finds that, for a single physical trajectory through the quark-gluon plasma, on the order of 100,000 trajectories are required to obtain sufficient statistics (see e.g. Figs. 5-7 where typical statistical uncertainties are reported). Finally, we note that one major advantage of the `QTraj` solver is that in this case one does not have to impose a cutoff on the orbital angular momentum l, whereas in the QuTiP 2 based implementation an angular momentum cutoff of $l_{\max} = 1$ was used. If one increases the angular momentum cutoff in the QuTiP 2 based code, the resulting run times become longer and memory limitations more severe, since the dimension of the reduced density matrix scales like $(2\,\mathrm{NUM}\,(l_{\max}+1))^2$, where the factor of 2 accounts for the color singlet and octet states.

Based on the performance data collected and discussions above, one can estimate

$$T_{\mathrm{qtraj}} \sim \frac{N_{\mathrm{traj}}\,\mathrm{NUM}\,\log \mathrm{NUM}}{N_{\mathrm{cores}}}\,, \tag{27}$$

where N_{cores} is the number of independent cores available and

$$T_{\mathrm{QuTiP\,2}}^{\mathrm{single\ core}} \sim (l_{\max}+1)^4\,\mathrm{NUM}^4\,. \tag{28}$$

As a case study, one can consider the case of $\mathrm{NUM} = 4096$ which was used in the phenomenological predictions contained in [44]. Assuming $N_{\mathrm{traj}} = 10^5$ and $l_{\max} = 1$, for a single core, one obtains $T_{\mathrm{qtraj}}^{\mathrm{single\ core}} \simeq 3 \times 10^5$ seconds, whereas the extrapolated prediction for $T_{\mathrm{QuTiP\,2}}^{\mathrm{single\ core}}$ is 3.6×10^7 seconds. Based on our fits, when $l_{\max} = 1$ and $N_{\mathrm{traj}} = 10^5$, the two codes become comparable in run time for lattices of size $\mathrm{NUM}_{\mathrm{cross}}^{l_{\max}=1} \sim 850$. For $l_{\max} \geq 1$, one can expect this point to be at $\mathrm{NUM}_{\mathrm{cross}} \sim 1700/(l_{\max}+1)$, which goes to zero as $l_{\max}$ becomes large. Summarizing, we find that for $\mathrm{NUM} \gtrsim \mathrm{NUM}_{\mathrm{cross}}$ `QTraj` running on a single core will be more computationally efficient than the QuTiP 2 implementation used in [24,25].

Finally, we emphasize that the above comparison is of the single core run time. Due to the embarrassingly parallel nature of the `QTraj` algorithm, one can simply run the code on many distributed cores simultaneously and no communication is required between cores. In principle, it is also possible to parallelize the QuTiP 2 `mesolve` based code; however, due to the fact that the computation does not easily factorize into independent processes, one can expect significant communication bottlenecks. Based on our results and the discussion above, our conclusion is that, when simulating large lattices, `QTraj` provides a superior implementation of the Lindblad equation solver, and it does so without imposing an angular momentum cutoff. We additionally note that, due to the portable and lightweight nature of the `QTraj` code, one can leverage state, national, and international computing facilities, where typically tens of thousands of cores are available, thus greatly accelerating the computation.

Eventually, one must consider how many trajectories are necessary to achieve a given target accuracy. We show results using different subsets of the same statistical ensemble of trajectories in Fig. 9 presenting S-wave overlaps in the left panel and off-diagonal overlaps in the right panel. The 1S survival probability is obtained to better than 10% accuracy already with approximately 10^4 trajectories, but the higher S-waves require at least a factor five more statistics and off-diagonal overlaps in general substantially more. Accessing the singlet P-wave from an initial singlet S-wave cannot be accomplished with fewer than three jumps; hence, it requires at

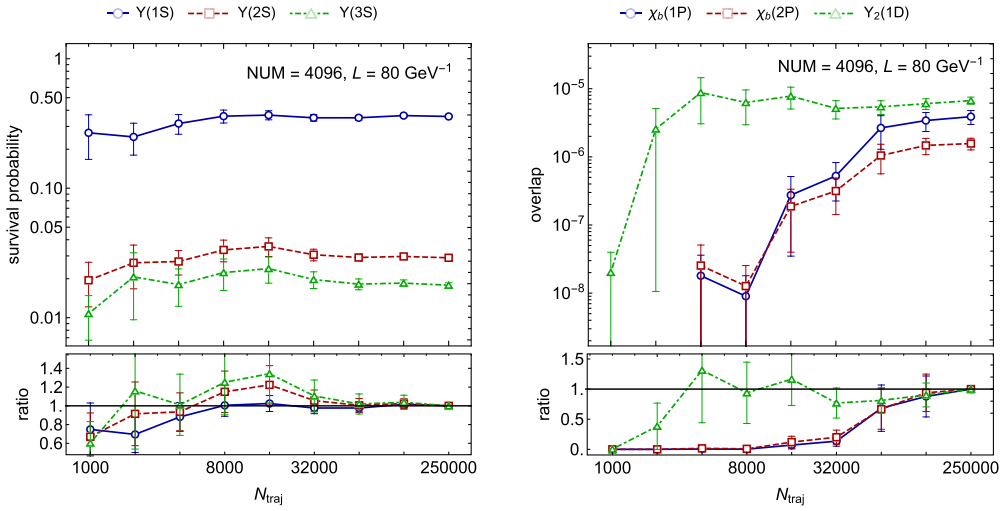


Fig. 9. (Left) Survival probabilities for $\Upsilon(1S)$, $\Upsilon(2S)$, and $\Upsilon(3S)$ versus the number of quantum trajectories for a fixed volume $L = 80$ GeV^{-1} and NUM $= 4096$. (Right) Off-diagonal overlaps with the $\chi_b(1P)$, $\chi_b(2P)$, and $\Upsilon(1D)$ states. For both panels, we used Gaussian S-wave initial conditions with $\Delta = 0.2a_0$ and Bjorken temperature evolution starting at $\tau = 0.6$ fm with $T_0 = 425$ MeV. Error bars indicate the statistical uncertainty associated with averaging the considered subset of quantum trajectories.

least two periods of evolution as an octet state between the jumps. For this reason, substantially more statistics are needed, and the final plateau may not be reached even with 2.5×10^5 trajectories, although the convergence is visible in the plot. The D-wave, which cannot be reached with fewer than two jumps, still appears to converge at a similar rate as the excited S-waves. Although these off-diagonal overlaps have substantially larger relative uncertainties than the S-waves, they are three orders of magnitude suppressed, rendering them quantitatively irrelevant.

9. Conclusions

In this paper, we introduce the `QTraj` code package allowing for the simulation of the quantum dynamics of heavy-quarkonium states in the quark-gluon plasma. In practice, the code uses a stochastic unraveling of the Lindblad equation called the quantum trajectories algorithm to reduce the solution of the in-medium 3D Lindblad equation to the solution of a one-dimensional Schrödinger equation along with stochastically sampled quantum jumps which can change the color and angular momentum quantum numbers of the state. The resulting 3D solution of the Lindblad equation allows one to compute the survival probabilities of heavy-quark bound states on a trajectory-by-trajectory basis. For the most recent physics application of `QTraj`, we refer the reader to [72], where phenomenological predictions for bottomonium suppression and elliptic flow in 5 TeV Pb-Pb collisions are presented.

This paper provides detailed documentation for downloading, compiling, and running `QTraj` in addition to providing theoretical background for the algorithm itself. We compare `QTraj` results obtained using a variety of lattice sizes, lattice spacings, temporal step sizes, etc., demonstrating that it is possible to achieve control of the various sources of systematic uncertainty. We also compare run times for the `QTraj` code to standard matrix-based solvers available in the QuTiP 2 package, finding that `QTraj` allows one to efficiently solve the Lindblad equation on large lattices without having to impose an angular momentum cutoff. The resulting code is embarrassingly parallel allowing the user to distribute the workload of generating independent quantum trajectories. Associated with this paper, we publicly release the `QTraj` code under a GPLv3 licence [48]. In doing so, we seek to assist other researchers studying open quantum systems approaches to heavy-quarkonium dynamics in the QGP and other physically or theoretically similar systems.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank N. Brambilla and A. Vairo for a very fruitful collaboration. M.A.E. received financial support from Xunta de Galicia (Centro singular de investigación de Galicia accreditation 2019-2022), the European Union ERDF, the “María de Maeztu” Units of Excellence program MDM2016-0692, the Spanish Research State Agency and from the European Research Council project ERC-2018-ADG-835105 YoctoLHC. M.S., A.I., and S.T. were supported by the U.S. Department of Energy, Office of Science, Office of Nuclear Physics Award No. DE-SC0013470. M.S., A.I., and S.T. also thank the Ohio Supercomputer Center for computational time under the auspices of Project No. PGS0253. J.H.W. was supported by the U.S. Department of Energy, Office of Science, Office of Nuclear Physics and Office of Advanced Scientific Computing Research within the framework of Scientific Discovery through Advance Computing (SciDAC) award Computing the Properties of Matter with Leadership Computing Resources. J.H.W.’s research was also funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - Projektnummer 417533893/GRK2575 “Rethinking Quantum Field Theory”. P.V. acknowledges support from the Bundesministerium für Bildung und Forschung project no. 05P2018 and the DFG cluster of excellence ORIGINS funded by the Deutsche Forschungsgemeinschaft under Germany’s Excellence Strategy - EXC-2094-390783311.

Appendix A. Packages required

In this appendix, we provide installation instructions for the `QTraj` package. The code has been tested on Mac OSX 10.15+, Ubuntu 18+, and Red Hat Enterprise Linux 7+. Below we provide installation instructions for the former two since these are more commonly used. To begin, we list the external packages required to compile and run `QTraj`. The first requirement is a modern C++

compiler, e.g. GNU g++ or LLVM. In addition to this, `QTraj` requires:

1. FFTW v3.3.8 or greater [63].
2. GNU GSL v2.6 or greater [73].
3. Intel MKL v2021.2 or greater [49] **OR** Armadillo 10.1.2 or greater [50].

Compilation and running of the unit tests associated with `QTraj` requires:

1. GoogleTest v1.10.0 or greater [74].
2. Cmake v3.19 [75] or greater is required for GoogleTest compilation.

A.1. Installation of FFTW

For Mac OSX, we recommend the installation of the Homebrew package manager [76]. Once this package manager is installed, the user should be able to execute the following commands in the terminal:

```
$ brew install fftw3
```

For Ubuntu, one can use `apt-get`:

```
$ sudo apt-get install fftw3-dev
```

A.2. Installation of GSL

For Mac OSX:

```
$ brew install gsl
```

For Ubuntu, one can use `apt-get`:

```
$ sudo apt-get install gsl-bin libgsl-dev
```

A.3. Installation of Intel MKL

To download Intel MKL, the user can follow the link in [49] to download and run the full installer for their operating system. This requires registering for an account with Intel. Once the installer has completed installation, the following line must be added to the end of the user's `.profile` or `.bash_profile` startup script; the terminal should be restarted afterwards.

```
source /opt/intel/oneapi/setvars.sh
```

If the MKL installation is in a non-standard location, the path to the `compilervars.sh` script appearing above needs to be adjusted appropriately.

A.4. Installation of Armadillo

For Mac OSX:

```
$ brew install armadillo
```

For Ubuntu, one can use `apt-get`:

```
$ sudo apt-get install liblapack-dev libblas-dev
libboost-dev libarmadillo-dev
```

A.5. Installation of GoogleTest

For Mac OSX:

```
$ brew install cmake
```

For Ubuntu, one can use:

```
$ sudo apt-get install cmake
```

Once `cmake` is installed, GoogleTest can be downloaded from [74], using

```
$ git clone https://github.com/google/googletest
```

or the user's preferred method. One must then execute:

```
$ cd googletest
$ mkdir build
$ cd build
$ cmake ..
$ make
$ sudo make install
```

Appendix B. Scripts for large scale deployment

The `scripts` directory in the main package contains Torque Portable Batch System (PBS) [77] scripts and HTCondor [78] scripts that can assist with large scale deployment of the code in supercomputing environments:

```
scripts
├── arrayJob.pbs
├── condorJob.sh
├── condor.job
├── scanImpactParameters.pbs
├── singleJob.pbs
└── manyTrajectories.pbs
```

Appendix C. Temperature file formats

`QTraj` v1.0 supports two temperature file formats. To activate these options, the user can set the `temperatureEvolution` parameter to 1 or 2. For both options, the file to load is specified by the `temperatureFile` parameter.

C.1. Trajectory-averaged temperature format

For option 1, the file to which `temperatureFile` points should have the following format

```
# t [fm/c]  T [GeV]  alphax  alphay  alphaz  lambda
     [GeV]
<NUMBER OF TAU POINTS>
<TAU_1> <T_1> <ALPHAX_1> <ALPHAY_1> <ALPHAZ_1> <
     LAMBDA_1>
<TAU_2> <T_2> <ALPHAX_2> <ALPHAY_2> <ALPHAZ_2> <
     LAMBDA_2>
  .       .       .           .           .
      .
  .       .
  .       .       .           .           .
      .
```

where TAU is the proper time τ, T is the temperature, ALPHA is $\alpha_{x,y,z}$ and LAMBDA is λ, a temperature-like scale which corresponds to temperature in the equilibrium limit. The scale parameters $\alpha_{x,y,z}$ appear in the anisotropic distribution function [79]

$$f(p) = f_{\text{iso}}\left(\frac{1}{\lambda}\sqrt{\sum_i \frac{p_i^2}{\alpha_i^2} + m^2}\right), \tag{C.1}$$

where $i \in \{x, y, z\}$ and f_{iso} is an isotropic distribution function which in thermal equilibrium is given by a Bose-Einstein distribution.

C.2. Trajectory-based temperature format

For option 2, the file to which `temperatureFile` points should have the following format

```
<NUMBER OF METADATA ELEMENTS>
<METADATA_1>
    .
    .
<METADATA_N>
<NUMBER OF TAU POINTS>
<TAU_1> <T_1>
<TAU_2> <T_2>
  .       .
  .       .
  .       .
```

As with option 1, τ is assumed to be in units of fm/c and T is assumed to be in units of GeV.

In practice, the metadata values are floating point numbers that represent data about the given trajectory which can be used in post-processing. Canonically, there are 6 metadata elements which record the following data for a given trajectory:

1. Impact Parameter in fm.
2. Initial production point x in fm.
3. Initial production point y in fm.
4. Initial point η (spatial rapidity).
5. Initial p_T in GeV.
6. Initial azimuthal angle ϕ.

Appendix D. Output file formats

D.1. Format of snapshot files

If the user sets the `saveWavefunctions` parameter to 1, snapshots of the wave-function are saved periodically to disk in files named `snapshot_<number>.tsv` where the number is the time step of the snapshot. The format of these files is as follows

```
<R_1> <ABS_PSI_1> <RE_PSI_1> <IM_PSI_1>
<R_2> <ABS_PSI_2> <RE_PSI_2> <IM_PSI_2>
  .
  .
  .
```

where `R` is the position in 1/GeV, `ABS_PSI` is $|\psi|$, `RE_PSI` is $\Re[\psi]$, and `IM_PSI` is $\Im[\psi]$.

D.2. Format of summary.tsv

If the user sets the `outputSummaryFile` variable to 1, `QTraj` outputs summary information concerning the evolution. Typically, this output is only used for debugging on a single quantum trajectory basis. The columns in this file correspond to

```
<TIME_1> <NORM_1> <COLOR_1> <L_1> <1S_1> <2S_1> <1
    P_1> <3S_1> <2P_1> <1D_1> <R_1>
<TIME_2> <NORM_2> <COLOR_2> <L_2> <1S_2> <2S_2> <1
    P_2> <3S_2> <2P_2> <1D_2> <R_2>
  .
  .
  .
```

where the following notation has been used:

1. `TIME` is the simulation time t in fm.
2. `NORM` is the norm of the wave-function $\langle\psi(t)|\psi(t)\rangle$.
3. `COLOR` is the color state; 0 = singlet, 1 = octet.
4. `L` is the angular momentum quantum number.
5. `1S`, `2S`, `1P` etc. are the quantum mechanical overlaps, e.g. $|\langle 1S|\psi(t)\rangle|^2$.
6. `R` is the expectation value of r normalized by the 1S expectation value of r, $\langle\psi(t)|r|\psi(t)\rangle / \langle 1S|r|1S\rangle$.

D.3. Format of ratios.tsv

If the user sets the `temperatureEvolution` parameter to 1 or 2, the output file `output/ratios.tsv` contains rows of data in the following format

```
<RAT_1S_1> <RAT_2S_1> <RAT_1P_1> <RAT_3S_1> <
    RAT_2P_1> <RAT_1D_1> <RAND_1> <INIT_L_1>
<RAT_1S_2> <RAT_2S_2> <RAT_1P_2> <RAT_3S_2> <
    RAT_2P_2> <RAT_1D_2> <RAND_2> <INIT_L_2>
  .
  .
  .
```

If the user sets the temperatureEvolution parameter to 3, then the output file `output/ratios.tsv` will contain rows of data in the following format

```
<IP_1> <X_1>  <Y_1> <ETA_1> <PT_1> <PHI_1>
<RAT_1S_1> <RAT_2S_1> <RAT_1P_1> <RAT_3S_1> <
    RAT_2P_1> <RAT_1D_1> <RAND_1> <INIT_L_1>
<IP_2> <X_2>  <Y_2> <ETA_2> <PT_2> <PHI_2>
<RAT_1S_2> <RAT_2S_2> <RAT_1P_2> <RAT_3S_2> <
    RAT_2P_2> <RAT_1D_2> <RAND_2> <INIT_L_2>
  .
  .
  .
```

where the following abbreviations have been used:

1. `RAT` is the overlap ratio or raw overlap depending on the state in consideration.[7]
2. `RAND` is the first random number generated during the evolution.
3. `INIT_L` is the initial value of angular momentum quantum number l.
4. `IP` is the impact parameter in fm.
5. `X`, `Y` are the initial production point (X, Y) (in fm) for the physical trajectory.
6. `ETA` is the spatial rapidity η.
7. `PT` is the initial transverse momentum p_T in GeV for the physical trajectory.
8. `PHI` is the initial azimuthal angle ϕ of the physical trajectory.

Appendix E. Supported potentials

E.1. Munich potential

From Eq. (16), we can write down the combined Hamiltonian [25] for singlet and octet states as

$$H = \begin{pmatrix} h_s + \frac{1}{2}\gamma r^2 & 0 \\ 0 & h_o + \frac{7}{32}\gamma r^2 \end{pmatrix}, \tag{E.1}$$

[7] Which output form is given for a given state depends on the initial overlap of the state. In the case that there is a zero overlap in the initial condition, `QTraj` records the final quantum mechanical overlap modulus squared, otherwise the ratio of the final to initial quantum mechanical overlaps is recorded.

where

$$h_s = \frac{\mathbf{p}^2}{m} - C_F \frac{\alpha_s}{r} = \frac{\mathbf{p}^2}{m} - \frac{\alpha}{r} \,, \tag{E.2}$$

$$h_o = \frac{\mathbf{p}^2}{m} + \frac{\alpha_s}{2N_c r} = \frac{\mathbf{p}^2}{m} + \frac{1}{8}\frac{\alpha}{r} \,. \tag{E.3}$$

with, $N_c = 3$, $C_F = \frac{N_c^2-1}{2N_c} = \frac{4}{3}$, and $\alpha = C_F\,\alpha_s = \frac{4}{3}\alpha_s$.

From Eq. (E.1), we can write down the singlet Hamiltonian as

$$H_s = h_s + \frac{1}{2}\gamma\, r^2 = \frac{\mathbf{p}^2}{m} - \frac{\alpha}{r} + \frac{1}{2}\hat{\gamma}\, T^3 r^2 \,, \tag{E.4}$$

and the octet Hamiltonian as

$$H_o = h_o + \frac{7}{32}\gamma\, r^2 = \frac{\mathbf{p}^2}{m} + \frac{1}{8}\frac{\alpha}{r} + \frac{7}{32}\hat{\gamma}\, T^3 r^2 \,, \tag{E.5}$$

where $\gamma = \hat{\gamma} T^3$ with $-3.5 < \hat{\gamma} < 0$ [44,66]. The factor $\frac{7}{32}$ arises as $\frac{1}{4}\frac{N_c^2-2}{N_c^2-1}$.

From Eq. (E.4) and Eq. (E.5), we extract the real parts of the singlet and octet potentials

$$V_R^s(r) = -\frac{\alpha}{r} + \frac{1}{2}\hat{\gamma}\, T^3 r^2 \,, \tag{E.6}$$

$$V_R^o(r) = \frac{1}{8}\frac{\alpha}{r} + \frac{7}{32}\hat{\gamma}\, T^3 r^2 \,. \tag{E.7}$$

We use Eq. (C3) and Eq. (C6) of [25] to write down the imaginary part of the singlet and octet potentials

$$V_I^s(r) = -\frac{1}{2}\kappa\, r^2 = -\frac{1}{2}\hat{\kappa}\, T^3 r^2 \,, \tag{E.8}$$

and

$$V_I^o(r) = -\frac{1}{4}\frac{N_c^2-2}{N_c^2-1}\kappa\, r^2 = -\frac{7}{32}\kappa\, r^2 = -\frac{7}{32}\hat{\kappa}\, T^3 r^2 \,, \tag{E.9}$$

where $\kappa = \hat{\kappa} T^3$. The next-to-leading-order perturbative expansion of $\hat{\kappa}$ is defined by Eq. (C4) of [25]. We use a parameterization of this result which is fitted to lattice data [67]. The resulting parameterization for $\hat{\kappa}$ is given by [25]

$$\hat{\kappa} = \begin{cases} \left(0.00397x^{3/2} - 0.08341x + 0.98031x^{1/2} - 0.51889\right)^{-1} \\ \text{in case of central limit fit} \\ \left(0.00712x^{3/2} - 0.14262x + 1.55601x^{1/2} - 0.80712\right)^{-1} \\ \text{in case of lower limit fit} \\ \left(0.00272x^{3/2} - 0.05854x + 0.71513x^{1/2} - 0.38197\right)^{-1} \\ \text{in case of upper limit fit} \end{cases} , \tag{E.10}$$

where $x = T/T_C$. In Fig. 10, we plot $\hat{\kappa}$ as a function of the temperature in units of T_c where T_c = 155 MeV, which is close to recent determinations in (2+1)-flavor lattice QCD with physical quark masses, e.g. [80–82].

The resulting complex-valued potentials for singlet and octet states are

$$V^s_{\text{Munich}}(r) = V_R^s(r) + iV_I^s(r) \,, \tag{E.11}$$

and

$$V^o_{\text{Munich}}(r) = V_R^o(r) + iV_I^o(r) \,. \tag{E.12}$$

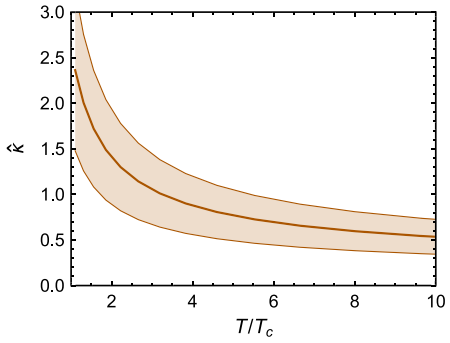


Fig. 10. $\hat{\kappa}$ as a function of the temperature in units of T_c, where T_c = 155 MeV; based on data from [67].

E.2. Isotropic KSU potential

For the KSU potential, we make use of a Karsch-Mehr-Satz type potential based on the internal energy [7,55,56]

$$V^{\text{iso}}_{\text{KMS}}(r) = -\frac{a}{r}(1 + m_D r)e^{-m_D r} + \frac{2\sigma}{m_D}\left[1 - e^{-m_D r}\right] - \sigma r e^{-m_D r} \,, \tag{E.13}$$

where $a = 0.409$ is the effective coupling, $\sigma = 0.21$ GeV^2 is the string tension, and the isotropic Debye mass is defined by

$$m_D(T) = F_{m_D} T \sqrt{N_c\left(1 + \frac{N_F}{6}\right)\pi\alpha} \,, \quad \alpha = \frac{4}{3}\alpha_s(2\pi T) \,, \tag{E.14}$$

where F_{m_D} is an adjustable factor called the m_D-factor and $\alpha_s(5\text{ GeV}) = 0.2034$ [83] is the strong coupling. Note that, in the current version of `QTraj`, the various parameters such as σ, etc. are hardcoded as defined in `potential.h`. Changing these parameters currently requires recompilation of the code.

For the imaginary part of the model potential, we use the result obtained from leading-order finite temperature perturbation theory [8] assuming the hierarchy $r \sim 1/m_D$ of the chromoelectric screening regime,

$$V_{\text{I}}^{\text{iso}}(r) = -\alpha_s C_F T \phi(\hat{r}) \,, \tag{E.15}$$

where $\hat{r} \equiv m_D r$ and

$$\phi(\hat{r}) \equiv 2\int_0^\infty dz \frac{z}{(z^2+1)^2}\left[1 - \frac{\sin(z\hat{r})}{z\hat{r}}\right]. \tag{E.16}$$

In vacuum, we take the heavy-quark potential to be given by a Cornell potential

$$V_{\text{vac}}(r) = \begin{cases} -\frac{a}{r} + \sigma r & \text{if } r \le r_{\text{SB}} \\ -\frac{a}{r_{\text{SB}}} + \sigma r_{\text{SB}} & \text{if } r > r_{\text{SB}} \end{cases} , \tag{E.17}$$

where $r_{\text{SB}} = 1.25$ fm is the finite string breaking distance. To match smoothly onto the zero temperature limit, we use

$$V_R^{\text{iso}}(r) = \begin{cases} V^{\text{iso}}_{\text{KMS}}(r) & \text{if } V^{\text{iso}}_{\text{KMS}}(r) \le V_{\text{vac}}(r_{\text{SB}}) \\ V_{\text{vac}}(r_{\text{SB}}) & \text{if } V^{\text{iso}}_{\text{KMS}}(r) > V_{\text{vac}}(r_{\text{SB}}) \end{cases} . \tag{E.18}$$

The resulting final isotropic complex-valued KSU potential is of the form

$$V^{\text{iso}}_{\text{KSU}}(r) = V_R^{\text{iso}}(r) + iV_I^{\text{iso}}(r) \,. \tag{E.19}$$

E.3. Anisotropic KSU potential

After considering the "minimal" extension [55] of the internal energy based isotropic KMS potential [7,56], we write down the real part of the anisotropic KMS potential

$$V_{\rm KMS}^{\rm aniso}(r) = -\frac{a}{r}(1+\mu r)e^{-\mu r} + \frac{2\sigma}{\mu}\left[1-e^{-\mu r}\right] - \sigma r e^{-\mu r}\,, \quad \text{(E.20)}$$

where μ is the anisotropic screening scale (anisotropic Debye mass) defined via

$$\mu^2(\xi, T) = f(\xi) m_D^2(T)\,. \quad \text{(E.21)}$$

The function $f(\xi)$ in Eq. (E.21) is given by [84]

$$f(\xi) = \frac{\sqrt{2}(1+\xi)\arcsin\left(\sqrt{\frac{\xi}{1+\xi}}\right)}{\sqrt{\xi}\sqrt{1+\xi+\frac{(1+\xi)^2\arcsin\left(\sqrt{\frac{\xi}{1+\xi}}\right)}{\sqrt{\xi}}}}\,, \quad \text{(E.22)}$$

which has the following properties

$$\lim_{\xi\to 0} f(\xi) = 1 - \frac{2}{45}\xi^2 + \frac{44}{945}\xi^3 - \frac{8}{189}\xi^4 + \mathcal{O}(\xi)^{9/2}\,, \quad \text{(E.23)}$$

and

$$\lim_{\xi\to\infty} f(\xi) = \sqrt{\pi}\left(\frac{1}{\xi}\right)^{1/4} - \frac{2}{\sqrt{\pi}}\left(\frac{1}{\xi}\right)^{3/4} + \frac{4}{3\sqrt{\pi}}\left(\frac{1}{\xi}\right)^{7/4} + \mathcal{O}\left(\frac{1}{\xi}\right)^{9/4}\,, \quad \text{(E.24)}$$

in the small and large-ξ limit respectively.

When ξ is negative, we use

$$f(\xi) = \frac{\sqrt{2}\,\mathrm{arcsinh}\left(\sqrt{\frac{|\xi|}{1-|\xi|}}\right)}{|\xi|^{1/4}\sqrt{\frac{\sqrt{|\xi|}}{1-|\xi|}+\mathrm{arcsinh}\left(\sqrt{\frac{|\xi|}{1-|\xi|}}\right)}}\,, \text{ when } \xi<0\,. \quad \text{(E.25)}$$

For the imaginary part of the potential, we use [85]

$$V_I^{\rm aniso}(r) = -\frac{\alpha_s C_F \lambda \phi(\hat{r})}{\sqrt{1+\frac{\xi}{3}}}\,, \quad \text{(E.26)}$$

with $\hat{r} = m_D(\lambda) r$ and $m_D(\lambda) = F_{m_D}\lambda\sqrt{N_c\left(1+\frac{N_F}{6}\right)\pi\alpha}$. The parameter λ is a temperature-like scale, see Appendix C.1 and, in particular, Eq. (C.1) for its functional definition which is based on a generalization of the momentum-anisotropic form introduced originally in [86] and extended to include multiple anisotropy parameters in [79,87].

To match smoothly onto the zero temperature limit, we use

$$V_R^{\rm aniso}(r) = \begin{cases} V_{\rm KMS}^{\rm aniso}(r) & \text{if } V_{\rm KMS}^{\rm aniso}(r) \le V_{\rm vac}(r_{\rm SB}) \\ V_{\rm vac}(r_{\rm SB}) & \text{if } V_{\rm KMS}^{\rm aniso}(r) > V_{\rm vac}(r_{\rm SB}) \end{cases}\,. \quad \text{(E.27)}$$

The resulting final anisotropic complex-valued KSU potential is of the form

$$V_{\rm KSU}^{\rm aniso}(r) = V_R^{\rm aniso}(r) + i V_I^{\rm aniso}(r)\,. \quad \text{(E.28)}$$

References

[1] A. Bazavov, J. Phys. Conf. Ser. 446 (2013) 012011.
[2] S. Borsanyi, EPJ Web Conf. 137 (2017) 01006.
[3] A. Bazavov, J.H. Weber, Prog. Part. Nucl. Phys. 116 (2021) 103823.
[4] R. Averbeck, J.W. Harris, B. Schenke, Heavy-Ion Physics at the LHC, Springer International Publishing, Cham, 2015, pp. 355–420.
[5] W. Busza, K. Rajagopal, W. van der Schee, Annu. Rev. Nucl. Part. Sci. 68 (2018) 339–376.
[6] T. Matsui, H. Satz, Phys. Lett. B 178 (1986) 416.
[7] F. Karsch, M.T. Mehr, H. Satz, Z. Phys. C 37 (1988) 617.
[8] M. Laine, O. Philipsen, P. Romatschke, M. Tassler, J. High Energy Phys. 03 (2007) 054.
[9] M. Strickland, Phys. Rev. Lett. 107 (2011) 132301.
[10] A. Rothkopf, Phys. Rep. 858 (2020) 1–117.
[11] Y. Akamatsu, Quarkonium in Quark-Gluon Plasma: Open Quantum System Approaches Re-examined, 2020.
[12] A. Rothkopf, Nucl. Phys. A 1005 (2021) 121922.
[13] N. Brambilla, J. Ghiglieri, A. Vairo, P. Petreczky, Phys. Rev. D 78 (2008) 014017.
[14] N. Brambilla, M.A. Escobedo, J. Ghiglieri, A. Vairo, J. High Energy Phys. 12 (2011) 116.
[15] N. Brambilla, M.A. Escobedo, J. Ghiglieri, A. Vairo, J. High Energy Phys. 05 (2013) 130.
[16] P. Braun-Munzinger, J. Stachel, Phys. Lett. B 490 (2000) 196–202.
[17] R.L. Thews, M. Schroedter, J. Rafelski, Phys. Rev. C 63 (2001) 054905.
[18] A. Emerick, X. Zhao, R. Rapp, Eur. Phys. J. A 48 (2012) 72.
[19] N. Borghini, C. Gombeaud, Eur. Phys. J. C 72 (2012) 2000.
[20] Y. Akamatsu, Phys. Rev. D 87 (2013) 045016.
[21] Y. Akamatsu, Phys. Rev. D 91 (2015) 056002.
[22] J.-P. Blaizot, D. De Boni, P. Faccioli, G. Garberoglio, Nucl. Phys. A 946 (2016) 49–88.
[23] R. Katz, P.B. Gossiaux, Ann. Phys. 368 (2016) 267–295.
[24] N. Brambilla, M.A. Escobedo, J. Soto, A. Vairo, Phys. Rev. D 96 (2017) 034021.
[25] N. Brambilla, M.A. Escobedo, J. Soto, A. Vairo, Phys. Rev. D 97 (2018) 074009.
[26] S. Kajimoto, Y. Akamatsu, M. Asakawa, A. Rothkopf, Phys. Rev. D 97 (2018) 014003.
[27] J.-P. Blaizot, M.A. Escobedo, J. High Energy Phys. 06 (2018) 034.
[28] Y. Akamatsu, M. Asakawa, S. Kajimoto, A. Rothkopf, J. High Energy Phys. 07 (2018) 029.
[29] X. Yao, T. Mehen, Phys. Rev. D 99 (2019) 096028.
[30] X. Yao, B. Müller, Phys. Rev. D 100 (2019) 014008.
[31] J.-P. Blaizot, M.A. Escobedo, Phys. Rev. D 98 (2018) 074007.
[32] X. Yao, T. Mehen, J. High Energy Phys. 21 (2020) 062.
[33] X. Yao, Open Quantum Systems for Quarkonia, 2021.
[34] V. Gorini, A. Kossakowski, E. Sudarshan, J. Math. Phys. 17 (1976) 821.
[35] G. Lindblad, Commun. Math. Phys. 48 (1976) 119.
[36] J. Johansson, P. Nation, F. Nori, Comput. Phys. Commun. 183 (2012) 1760–1772.
[37] J. Johansson, P. Nation, F. Nori, Comput. Phys. Commun. 184 (2013) 1234–1240.
[38] J. Dalibard, Y. Castin, K. Molmer, Phys. Rev. Lett. 68 (1992) 580–583.
[39] K. Mølmer, Y. Castin, J. Dalibard, J. Opt. Soc. Am. B 10 (1993) 524–538.
[40] M.B. Plenio, P.L. Knight, Rev. Mod. Phys. 70 (1998) 101–144.
[41] H. Carmichael, Statistical Methods in Quantum Optics: Master Equations and Fokker-Planck Equations, Springer, New York, 1999.
[42] U. Weiss, Quantum Dissipative Systems, World Scientific, 1993.
[43] A.J. Daley, Adv. Phys. 63 (2014) 77–149.
[44] N. Brambilla, M.A. Escobedo, M. Strickland, A. Vairo, P. Vander Griend, J.H. Weber, J. High Energy Phys. 05 (2021) 136.
[45] J. Boyd, T. Cook, A. Islam, M. Strickland, Phys. Rev. D 100 (2019) 076019.
[46] A. Islam, M. Strickland, Phys. Lett. B 811 (2020) 135949.
[47] A. Islam, M. Strickland, J. High Energy Phys. 03 (2021) 235.
[48] Strickland, et al., QTraj 1.0 - Public Repository, https://bitbucket.org/michael_strickland/qtraj-fftw, 2021.
[49] Intel Corporation, Intel Math Kernel Library (Part of oneAPI), https://software.intel.com/content/www/us/en/develop/tools/oneapi/base-toolkit/download.html, 2021.
[50] Armadillo contributors, Armadillo: C++ library for linear algebra and scientific computing, http://arma.sourceforge.net/download.html, 2020.
[51] W.E. Caswell, G.P. Lepage, Phys. Lett. B 167 (1986) 437–442.
[52] G.T. Bodwin, E. Braaten, G.P. Lepage, Phys. Rev. D 51 (1995) 1125–1171, Phys. Rev. D 55 (1997) 5853, Erratum.
[53] A. Pineda, J. Soto, Nucl. Phys. B, Proc. Suppl. 64 (1998) 428–432.
[54] N. Brambilla, A. Pineda, J. Soto, A. Vairo, Nucl. Phys. B 566 (2000) 275.
[55] A. Dumitru, Y. Guo, A. Mocsy, M. Strickland, Phys. Rev. D 79 (2009) 054019.
[56] M. Strickland, D. Bazow, Nucl. Phys. A 879 (2012) 25–58.
[57] H. Breuer, F. Petruccione, The theory of open quantum systems, 2002.
[58] N. Gisin, I.C. Percival, J. Phys. A 26 (1993) 2233–2244.
[59] T. Miura, Y. Akamatsu, M. Asakawa, A. Rothkopf, Phys. Rev. D 101 (2020) 034011.
[60] R. Sharma, A. Tiwari, Phys. Rev. D 101 (2020) 074004.
[61] B. Fornberg, G. Whitham, Philos. Trans. R. Soc. 289 (1978) 373.

[62] T.R. Taha, M.I. Ablowitz, J. Comput. Phys. 55 (1984) 203–230.
[63] FFTW contributors, FFTW: a C subroutine library for computing the discrete Fourier transform, http://www.fftw.org/, 2020.
[64] J. Casalderrey-Solana, D. Teaney, Phys. Rev. D 74 (2006) 085012.
[65] S. Caron-Huot, G.D. Moore, Phys. Rev. Lett. 100 (2008) 052301.
[66] N. Brambilla, M.A. Escobedo, A. Vairo, P. Vander Griend, Phys. Rev. D 100 (2019) 054025.
[67] N. Brambilla, V. Leino, P. Petreczky, A. Vairo, Phys. Rev. D 102 (2020) 074503.
[68] W. R. Inc., Mathematica, Version 12.1, https://www.wolfram.com/mathematica, 2020.
[69] R. Pordes, D. Petravick, B. Kramer, D. Olson, M. Livny, A. Roy, P. Avery, K. Blackburn, T. Wenaus, F. Würthwein, I. Foster, R. Gardner, M. Wilde, A. Blatecky, J. McGee, R. Quick, The open science grid, J. Phys. Conf. Ser. 78 (2007) 012057, https://doi.org/10.1088/1742-6596/78/1/012057.
[70] I. Sfiligoi, D.C. Bradley, B. Holzman, P. Mhashilkar, S. Padhi, F. Wurthwein, in: 2009 WRI World Congress on Computer Science and Information Engineering, vol. 2, 2009, pp. 428–432.
[71] Strickland, et al., Sample trajectories for use in QTraj 1.0, http://personal.kent.edu/~mstrick6/code, 2021.
[72] N. Brambilla, M.A. Escobedo, M. Strickland, A. Vairo, P.V. Griend, J.H. Weber, Bottomonium production in heavy-ion collisions using quantum trajectories: differential observables and momentum anisotropy, TUM-EFT 147/21, HU-EP-21/18-RTG, arXiv:2107.0622, 2021.
[73] Free Software Foundation, GNU Scientific Library, https://www.gnu.org/software/gsl/, 2019.
[74] Google, Inc., GoogleTest, https://github.com/google/googletest, 2020.
[75] Kitware, Inc. and contributors, Cmake, https://cmake.org/download/, 2020.
[76] Homebrew contributors, Homebrew, https://brew.sh, 2020.
[77] Veridian Information Solutions, OpenPBS, https://www.openpbs.org/, 2000.
[78] HTCondor team, HTCondor, https://research.cs.wisc.edu/htcondor/, 2021.
[79] M. Nopoush, R. Ryblewski, M. Strickland, Phys. Rev. C 90 (2014) 014908.
[80] A. Bazavov, N. Brambilla, H.T. Ding, P. Petreczky, H.P. Schadler, A. Vairo, J.H. Weber, Phys. Rev. D 93 (2016) 114502.
[81] A. Bazavov, et al., HotQCD, Phys. Lett. B 795 (2019) 15–21.
[82] S. Borsanyi, Z. Fodor, J.N. Guenther, R. Kara, S.D. Katz, P. Parotto, A. Pasztor, C. Ratti, K.K. Szabo, Phys. Rev. Lett. 125 (2020) 052001.
[83] C. McNeile, C.T.H. Davies, E. Follana, K. Hornbostel, G.P. Lepage, Phys. Rev. D 82 (2010) 034512.
[84] M. Strickland, J. High Energy Phys. 12 (2018) 128.
[85] A. Islam, M. Strickland, Including momentum-space anisotropy in real-time quantum mechanical calculations of quarkonium suppression, forthcoming.
[86] P. Romatschke, M. Strickland, Phys. Rev. D 68 (2003) 036004.
[87] M. Martinez, R. Ryblewski, M. Strickland, Phys. Rev. C 85 (2012) 064913.